

Graduation Project
AI-Powered Network Attack Detection System and Analyst Dashboard

Prepared by:

Göktuğ VARAN 210408029
Mehmet Taha YILMAZ 210408039

Project Advisor

Prof. Dr. Özgür Koray ŞAHİNGÖZ

Department of Computer Engineering
Faculty of Engineering and Natural Sciences

2026-Spring

ÖZET

Bu çalışma, yapay zekâ destekli ağ saldırı tespit sistemi ve analist paneli geliştir-meyi amaçla-maktadır. Projenin temel amacı, makine öğrenmesi tabanlı bir saldırı tespit prototipi oluşturu-rarak ağ trafiğini sınıflandırmak, zararlı aktiviteleri tespit etmek, saldırı türlerini belirlemek ve sonuçları analist odaklı bir arayüz üzerinden görselleştirmektir.

Önerilen sistem iki ana bölümden oluşmaktadır: makine öğrenmesi süreci ve yazılım uygu-laması. Makine öğrenmesi sürecinde ağ trafiği verileri temizlenmiş, ön işleme tabi tutulmuş, analiz edilmiş ve özellik mühendisliği ile veri sızıntısı analizi adımlarından geçirilmiştir. Bi-nary ve multiclass sınıflandırma modelleri Random Forest, XGBoost ve CatBoost gibi ağaç ta-banlı makine öğrenmesi algoritmaları kullanılarak geliştirilmiştir. Modeller accuracy, weighted F1-score, macro F1-score, ROC-AUC, confusion matrix ve sınıf bazlı F1-score metrikleriyle değerlendirilmiştir.

Deneysel sonuçlar, önerilen yaklaşımın per-file değerlendirme senaryosunda güçlü perfor-mans elde ettiğini göstermiştir. Binary sınıflandırma modeli benign ve zararlı trafiği başarılı şekilde ayırt ederken, multiclass model çoğu büyük saldırı kategorisinde yüksek performans göstermiştir. Ancak Web Attack - XSS ve Web Attack - Brute Force gibi az örnekli web saldırısı sınıflarında daha düşük performans gözlemlenmiştir. Bu durum, saldırı tespit problemlerinde sınıf bazlı değerlendirmenin önemini göstermektedir.

Ayrıca proje kapsamında merkezi modelleme yaklaşımına ek olarak FedAvg tabanlı bir fed-erated learning simülasyonu da uygulanmıştır. Bu simülasyonda beş istemci üzerinde yerel eğitim yapılmış, beş iletişim turu sonunda model ağırlıkları FedAvg yöntemiyle birleştirilmiş ve global model holdout test verisi üzerinde değerlendirilmiştir. Bu çalışma, ham verinin istem-ciler arasında paylaşılmadığı dağıtık öğrenme yaklaşımının proje kapsamında deneysel olarak incelendiğini göstermektedir.

Seçilen Random Forest modelleri dışa aktarılmış ve FastAPI tabanlı backend sistemine en-tegre edilmiştir. React/Next.js ile geliştirilen analist paneli; tahmin sonuçlarını, trafik akışını, saldırı türü dağılımını, son güvenlik olaylarını ve SHAP tabanlı açıklanabilirlik bilgilerini görsel-leştirmektedir. Geliştirilen sistem, makine öğrenmesi tabanlı saldırı tespitinin çevrimdışı bir modelden çalışan bir yazılım prototipine dönüştürülebileceğini göstermektedir. Gelecek çalış-malarda canlı paket yakalama, kalıcı veritabanı kullanımı, model drift takibi, SIEM entegrasy-onu ve üretim ortamına daha yakın dağıtım süreçleri geliştirilebilir.

Anahtar Kelimeler: ağ saldırı tespiti, makine öğrenmesi, siber güvenlik, ağ trafiği sınıflandırma, açıklanabilir yapay zekâ, güvenlik analist paneli

ABSTRACT

This study presents an AI-powered network attack detection system and analyst dashboard. The main objective of the project is to develop a machine learning-based intrusion detection prototype that can classify network traffic, detect malicious activities, identify attack types, and visualize the results through an analyst-oriented interface.

The proposed system consists of two main parts: a machine learning pipeline and a software implementation. In the machine learning pipeline, network traffic data was cleaned, pre-processed, analyzed, and transformed through feature engineering and data leakage analysis. Binary and multiclass classification models were developed using tree-based machine learning algorithms, including Random Forest, XGBoost, and CatBoost. The models were evaluated using accuracy, weighted F1-score, macro F1-score, ROC-AUC, confusion matrices, and per-class F1-score analysis.

The experimental results showed that the proposed approach achieved strong performance in the per-file evaluation scenario. The binary classification model successfully distinguished benign and malicious traffic, while the multiclass model achieved high performance for most major attack categories. However, lower performance was observed for minority web attack classes, which shows the importance of per-class evaluation in intrusion detection problems.

In addition to the centralized modelling approach, a FedAvg-based federated learning simulation was also implemented. In this simulation, local training was performed across five clients, model parameters were aggregated over five communication rounds using FedAvg, and the final global model was evaluated on the holdout test set. This extension demonstrates the feasibility of distributed model training without sharing raw client data.

The selected Random Forest models were exported and integrated into a FastAPI backend. A React/Next.js analyst dashboard was developed to visualize prediction results, traffic flow, attack distribution, recent alerts, and SHAP-based explanation information. The system demonstrates how machine learning-based intrusion detection can be transformed from an offline model into a working software prototype. Future improvements may include live packet capture, persistent database storage, model drift monitoring, SIEM integration, and deployment in a production-like environment.

Keywords: network intrusion detection, machine learning, cybersecurity, network traffic classification, explainable AI, security analyst dashboard

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project advisor, Prof. Dr. Özgür Koray ŞAHİNGÖZ, for his valuable guidance, support, and understanding throughout every stage of this graduation project. His feedback and direction greatly contributed to the development and completion of this study.

We would also like to thank our families for their continuous support, encouragement, and patience throughout our academic journey. Their belief in us has been a constant source of motivation.

Finally, we would like to thank all our friends who supported us, motivated us, and stood by us during this process. Their encouragement helped us complete this project with greater confidence and determination.

ÖZET	iii
ABSTRACT	v
ACKNOWLEDGEMENTS	vii
TABLE OF CONTENTS	ix
LIST OF TABLES	xi
LIST OF FIGURES.....	xiii
SYMBOLS AND ABBREVIATIONS.....	xv
1 INTRODUCTION	1
1.1 Problem Statement	1
1.2 Project Purpose.....	1
1.3 Project Scope	2
1.4 Project Objective and Success Criteria	2
1.5 Report Outline	4
2 RELATED WORK	5
2.1 Existing Systems	6
2.2 General Problems of Existing Systems	7
2.3 Comparison Between Current and Proposed Method	8
3 METHODOLOGY	11
3.1 Needs Analysis.....	11
3.2 Dataset Overview	14
3.3 Data Preprocessing and Cleaning	15
3.4 Feature Engineering	16
3.5 Data Leakage Analysis	19
3.6 Model Development	22
3.7 Model Evaluation and Cross-Validation	23

3.8	System Architecture	25
3.9	Backend API Implementation	27
3.10	Analyst Dashboard Design	29
3.11	Explainability Module	31
3.12	Federated Learning Simulation.....	31
3.13	Testing	33
4	EXPERIMENTAL RESULTS	36
4.1	Final Binary Classification Performance	36
4.2	Multiclass Classification Results.....	37
4.3	Model Comparison Results.....	38
4.4	Per-Class F1-Score Analysis	39
4.5	Dashboard Output Results.....	40
4.6	Summary of Experimental Findings	40
4.7	Discussion of Results	41
5	CONCLUSION.....	45
	REFERENCES	48
	DECLARATION	50

LIST OF TABLES

Table 1.1: Project Objectives and Success Criteria	3
Table 2.1: Comparison Between Existing Systems and Proposed Method	9
Table 3.1: Functional and Non-Functional Requirements	13
Table 3.2: Leakage-Oriented Feature Analysis	21
Table 3.3: Evaluation Metrics Used in the Project	25
Table 3.4: Backend API Endpoints	28
Table 3.5: FedAvg Federated Learning Results	32
Table 3.6: System Testing Summary	34
Table 4.1: Binary Classification Performance Comparison	36
Table 4.2: Multiclass Classification Performance Comparison	38
Table 4.3: Limitations and Future Improvements	44

LIST OF FIGURES

Figure 3.1: General Methodology Workflow	11
Figure 3.2: Class Distribution Before Controlled Sampling	14
Figure 3.3: Class Distribution Before and After Controlled Sampling	15
Figure 3.4: Class Distribution After Cleaning	16
Figure 3.5: Features with High Zero Ratio.....	17
Figure 3.6: Mutual Information-Based Feature Importance.....	18
Figure 3.7: Skewness Analysis of Features	19
Figure 3.8: Model Performance Before and After Feature Engineering.....	19
Figure 3.9: Model Comparison Overview	23
Figure 3.10: Proposed System Architecture	26
Figure 3.11: Analyst Dashboard Overview.....	30
Figure 3.12: Alert Details and XAI Explanation	30
 Figure 4.1: Binary Classification Confusion Matrix.....	 37
Figure 4.2: Multiclass Classification Confusion Matrix.....	38
Figure 4.3: Model Performance by Evaluation Scenario	39
Figure 4.4: Per-Class F1-Score Comparison	39
Figure 4.5: Dashboard Visualization of Detection Results During Real-Time-Like Monitoring	40

SYMBOLS AND ABBREVIATIONS

ANN: Artificial Neural Network

API: Application Programming Interface

APWG: Anti-Phishing Working Group

CICIDS: Canadian Institute for Cybersecurity Intrusion Detection Dataset

CNN: Convolutional Neural Network

CORS: Cross-Origin Resource Sharing

DDoS: Distributed Denial of Service

DGA: Domain Generation Algorithm

DL: Deep Learning

DNN: Deep Neural Network

DNS: Domain Name System

DoS: Denial of Service

DT: Decision Tree

ET: Extra Trees

FedAvg: Federated Averaging

FL: Federated Learning

HTTP: Hypertext Transfer Protocol

HTTPS: Hypertext Transfer Protocol Secure

IDN: Internationalized Domain Name

IDS: Intrusion Detection System

IoT: Internet of Things

IP: Internet Protocol

KNN: K-Nearest Neighbors

LGBM: Light Gradient Boosting Machine

LSTM: Long Short-Term Memory

ML: Machine Learning

MLP: Multi-Layer Perceptron

NLP: Natural Language Processing

QR: Quick Response

RF: Random Forest

RNN: Recurrent Neural Network

SHAP: SHapley Additive exPlanations

SIEM: Security Information and Event Management

SVM: Support Vector Machine

TLD: Top-Level Domain

URL: Uniform Resource Locator

XAI: Explainable Artificial Intelligence

XGB: XGBoost

1. INTRODUCTION

1.1 Problem Statement

With the rapid growth of computer networks, cloud-based services, and internet-connected systems, network security has become a critical requirement for modern information systems. Organizations continuously generate large volumes of network traffic, and this traffic may contain both normal user activities and malicious behaviors such as Distributed Denial of Service attacks, port scanning, brute-force attempts, and web-based attacks. Detecting these threats accurately and quickly is essential for protecting system availability, data confidentiality, and service reliability [1], [2], [3].

Traditional intrusion detection systems generally rely on predefined signatures or manually created rules. Although these systems can be effective against known attacks, they may be insufficient when attack patterns change or when previously unseen threats occur. In addition, rule-based systems often require continuous manual updates and may generate a high number of false alarms. For this reason, machine learning-based intrusion detection has become an important research area, since such models can learn behavioral patterns from network traffic data and classify suspicious activities automatically [4], [5], [6].

However, building an accurate machine learning model alone is not enough for a practical intrusion detection system. The detection results should also be presented to security analysts in a clear, understandable, and usable way. Analysts need to monitor recent traffic events, observe attack distributions, inspect critical alerts, and understand the possible reasons behind model predictions. Therefore, this project focuses not only on developing a machine learning-based network attack detection model, but also on integrating the model with an API and an analyst dashboard [3], [7].

In this project, an AI-powered network attack detection system and analyst dashboard is developed. The system analyzes network traffic records, predicts whether the traffic is benign or malicious, classifies the attack type when necessary, and visualizes the results through a dashboard designed for security monitoring.

1.2 Project Purpose

The main purpose of this project is to design and implement an AI-powered intrusion detection prototype that can classify network traffic by using machine learning techniques and present the detection results through an analyst dashboard [8], [9].

The project aims to combine three main components: a machine learning model, a backend

API, and a frontend dashboard. The machine learning component is responsible for learning attack patterns from network traffic data and producing binary and multiclass predictions. The backend API serves the trained models and provides prediction, history, and statistical data to the frontend. The dashboard visualizes the prediction results in a way that can support security analysts during monitoring and decision-making [8], [9].

Another important purpose of the project is to evaluate the reliability of the model by considering common problems in intrusion detection datasets, such as class imbalance, redundant features, highly skewed features, and possible data leakage. In this context, feature engineering, leakage analysis, and model comparison steps are included in the project workflow [7], [10].

The final goal is to create a working prototype that demonstrates how machine learning-based attack detection can be integrated with a software system and transformed into an interactive monitoring tool.

1.3 Project Scope

The scope of this project includes the development of a machine learning-based intrusion detection pipeline and its integration with a real-time-like dashboard prototype. The project starts with dataset preparation and continues with data cleaning, feature analysis, feature engineering, model training, model evaluation, backend development, and frontend dashboard implementation.

Within the machine learning part, network traffic records are analyzed and used to train classification models. Both binary classification and multiclass classification tasks are considered. Binary classification is used to distinguish benign traffic from attack traffic, while multiclass classification is used to identify the specific attack category. Different machine learning models are compared, and the final trained models are exported for use in the backend system.

Within the software implementation part, a backend API is developed to load the trained models and serve prediction-related data. The frontend dashboard is designed to display live traffic events, traffic statistics, attack distributions, alert records, and explainability information. The system is implemented as a prototype that simulates real-time traffic monitoring by using preprocessed network traffic records.

The project does not include live packet capture from a real network interface or deployment in a production security environment. Instead, it focuses on demonstrating the complete workflow of an AI-powered intrusion detection system, from data processing and model development to API integration and dashboard visualization.

1.4 Project Objective and Success Criteria

The main objectives of this project are to develop a machine learning-based network attack detection model, integrate it with a backend API, and visualize the detection results through an analyst dashboard. The success of the project is evaluated based on both machine learning

performance and software functionality.

Table 1.1: Project Objectives and Success Criteria

Description	Success Criteria
Develop a binary classification model	The model can classify network traffic as benign or attack with high accuracy and F1 score values
Develop a multiclass classification model	The model can identify different attack categories and achieve acceptable class-wise performance
Perform data cleaning and preprocessing	Missing values, duplicates, inconsistent labels, and unnecessary columns are handled before model training
Analyze feature quality	Highly correlated, duplicate, or problematic features are identified and handled during feature engineering
Analyze possible data leakage	Address-based or dataset-specific features that are considered as leakage-sensitive are evaluated and excluded
Compare different machine learning models	Multiple models such as Random Forest, XGBoost, and CatBoost are trained and compared using common classification metrics
Export the final trained model	The selected binary and multiclass models are saved and loaded correctly by the back-end API
Develop a backend API	The API successfully loads the trained models and provides prediction, history, and statistical information to the dashboard
Develop an analyst dashboard	The dashboard can display live events, traffic statistics, attack distribution, alert timeline, and alert details
Provide explainability support	The system can show important feature-based SHAP explanations to help analysts interpret model predictions

These objectives ensure that the project is evaluated not only as a machine learning study but also as a complete software-based intrusion detection prototype.

1.5 Report Outline

The rest of this report is organized as follows.

Chapter 2 presents the related studies and existing intrusion detection approaches. It discusses traditional signature-based systems, anomaly-based systems, and machine learning-based intrusion detection methods. It also explains the general limitations of existing systems and compares them with the proposed system.

Chapter 3 describes the methodology of the project. This chapter explains the dataset, pre-processing steps, feature engineering process, leakage analysis, model development, backend API implementation, dashboard design, and testing procedure.

Chapter 4 presents the experimental results and discussion. The performance of the trained models is evaluated using classification metrics, confusion matrices, model comparison results, and per-class performance analysis. The findings are interpreted and discussed in terms of model effectiveness, limitations, and practical applicability. Dashboard outputs are also presented to demonstrate the practical use of the developed system.

Chapter 5 concludes the report by summarizing the main outcomes of the project, discussing its contributions and limitations, and presenting possible future improvements such as live packet capture integration, production deployment, SIEM integration, and advanced alert management.

2. RELATED WORK

Intrusion Detection Systems have been widely studied in cybersecurity because network attacks are becoming more complex, frequent, and difficult to detect using only traditional rule-based techniques [1], [2], [3]. Existing studies generally focus on detecting malicious traffic using signature-based, anomaly-based, machine learning-based, deep learning-based, federated learning-based, and explainable AI-supported approaches.

Traditional signature-based intrusion detection systems compare network traffic with pre-defined attack signatures. These systems are useful for detecting known attacks, but they are limited when new or modified attack patterns appear. For this reason, anomaly-based IDS approaches have become more important. Anomaly-based systems learn normal network behavior and detect traffic that deviates from this normal pattern. This makes them more suitable for detecting unknown or zero-day attacks [11].

Recent studies show that machine learning and deep learning techniques are commonly used for network intrusion detection [1], [2]. Algorithms such as Random Forest, Support Vector Machine, Decision Tree, XGBoost, CatBoost, CNN, LSTM, autoencoders, and transformer-based models have been applied to benchmark intrusion detection datasets [4], [5]. For example, transformer-based IDS studies use self-attention mechanisms to learn relationships between network traffic features and improve classification performance on high-dimensional datasets. RTIDS, a robust transformer-based IDS, uses feature representation learning and self-attention to classify attacks on CICIDS2017 and CIC-DDoS2019 datasets [12]. The study reports strong results but also states that deep learning models may be expensive to train and that quick-response intrusion detection remains a challenge.

Another research direction is federated learning-based IDS. Federated learning allows multiple devices or gateways to train a shared model without transferring raw data to a central server [13]. This is especially important for IoT environments where privacy, scalability, and communication overhead are major concerns. FLUIDS combines federated learning with semi-supervised learning by using unlabeled data on IoT gateways and labeled data on a server [14]. This approach aims to improve privacy, training efficiency, and deployment cost. Similarly, federated deep learning studies for IoT intrusion detection show that centralized IDS architectures may create privacy risks, network overhead, and scalability problems, while federated learning can reduce raw data transfer and preserve data locality [15]. However, these studies also show that non-IID data distribution can reduce model performance in realistic IoT environments.

Explainable AI has also become an important topic in IDS research. Many machine learning and deep learning models can achieve high performance, but their decisions are often difficult

to interpret. This creates a trust problem for security analysts. Kernel SHAP-based anomaly detection studies show that XAI methods can be used to explain why a network instance is classified as anomalous [16]. One study used kernelSHAP with an autoencoder on the CICIDS2017 dataset and selected important features based on Shapley values to improve the anomaly detection model. This is directly related to the explainability part of the proposed project because our system also aims to show feature-based explanations to the analyst instead of presenting only the final prediction label.

2.1 Existing Systems

Existing intrusion detection systems can be grouped into several main categories. The first group is signature-based IDS. These systems detect attacks by comparing network traffic with known attack signatures. They are effective against previously observed threats, but they cannot easily detect new, modified, or zero-day attacks [11].

The second group is anomaly-based IDS. These systems learn normal network traffic behavior and classify unusual behavior as suspicious. Machine learning and deep learning models are widely used in anomaly-based IDS because they can learn complex traffic patterns from large datasets [1], [2]. Studies using CICIDS2017, NSL-KDD, UNSW-NB15, CIC-DDoS2019, TON-IoT, and other benchmark datasets show that ML and DL models can achieve high detection performance under controlled experimental conditions [4], [5].

Tree-based machine learning models such as Random Forest, XGBoost, and CatBoost are frequently used in IDS because they work well on tabular flow-based network data [4], [5]. These models can learn nonlinear relationships between traffic features and attack labels. They are also easier to train and deploy compared to very large deep learning architectures. For this reason, tree-based models remain practical choices for prototype-level intrusion detection systems [8].

Deep learning-based IDS approaches use models such as CNN, LSTM, RNN, autoencoders, and transformers [1], [2]. These models are useful for learning complex patterns in high-dimensional traffic data. For example, RTIDS uses a transformer-based architecture with self-attention to detect intrusions and improve feature representation learning [12]. The system includes data preparation, model construction, and real-time intrusion detection modules, showing how deep learning can be structured as an IDS framework.

Federated learning-based IDS systems are another important direction, especially for IoT and distributed environments [13], [15]. These systems train models collaboratively without sharing raw local data. FLUIDS is an example of a federated semi-supervised IDS that uses autoencoders on IoT gateways and a supervised classifier on a central server [14]. This structure is useful when labels are limited and privacy must be protected.

Explainable IDS systems focus on making model outputs understandable for human analysts [16]. Kernel SHAP-based approaches show that feature contribution values can be used to explain anomalous network traffic. This is important because security analysts need to un-

derstand not only whether an event is malicious, but also why the model produced that decision. Lightweight and edge-based IDS approaches further aim to reduce computational cost for resource-constrained environments [17].

2.2 General Problems of Existing Systems

Although existing IDS studies show promising results, several common problems remain. One of the most important problems is dataset quality. Many IDS models are evaluated on benchmark datasets, but the reliability of these datasets directly affects the validity of the reported results. Studies on CICIDS2017 show that dataset creation issues such as flow construction errors, feature extraction problems, labeling mistakes, and meaningless artifacts can affect machine learning evaluation [7], [10]. One study found that more than 25% of all flows in CICIDS2017 were meaningless artifacts, and for some attack categories this ratio reached around 50% [10]. This shows that high model performance on a benchmark dataset does not always guarantee reliable real-world performance.

Another problem is class imbalance. In intrusion detection datasets, benign traffic usually has a much larger number of samples than attack traffic. Some attack classes, especially web attacks, infiltration, SQL injection, or Heartbleed-like rare attacks, may have very few samples. This imbalance can cause the model to perform well on majority classes but poorly on minority attack classes [3], [18]. Deep learning studies on imbalanced IDS datasets state that accuracy alone may be misleading because minority attacks can remain undetected even when the overall accuracy is high [18].

A third problem is data leakage and shortcut learning. Some features may allow the model to learn dataset-specific patterns instead of general attack behavior. For example, timestamp, IP address, source port, destination port, or other environment-specific attributes may cause the model to memorize how the dataset was generated [7], [10]. A CICIDS2017 analysis specifically warns that source/destination IP, source port, destination port, timestamp, and similar attributes may create shortcut learning if they are used without careful analysis [7]. This problem is highly relevant to our project because we also performed leakage-oriented feature analysis and removed leakage-sensitive features.

Another limitation is generalization. Many IDS models perform very well when training and testing data come from similar distributions, but their performance decreases when tested on a different time period, different network environment, or different dataset [6], [7], [10]. This means that IDS models should be evaluated not only with random or stratified splits, but also with more realistic evaluation scenarios.

Interpretability is also a major problem. Many ML and DL-based IDS models produce only a final prediction label. However, analysts need to understand the reason behind an alert. Without explainability, IDS outputs may be difficult to trust and investigate [16]. XAI studies emphasize that explanation and interpretation are important for transforming black-box models into more transparent and accountable systems.

Finally, many existing studies focus mainly on offline model performance. They report accuracy, precision, recall, F1-score, and confusion matrices, but they do not always provide a working software prototype [8], [9]. In practice, an IDS should not only detect attacks but also deliver predictions through an API, store recent alerts, visualize traffic behavior, and support analyst interpretation. This gap between notebook-based model evaluation and practical deployment is one of the main motivations of our project.

2.3 Comparison Between Current and Proposed Method

Existing IDS studies provide valuable approaches for attack detection, but many of them focus only on model training and offline evaluation [1], [2]. In contrast, the proposed project aims to develop an end-to-end intrusion detection prototype that combines machine learning, backend API implementation, analyst dashboard visualization, and explainability support [8], [9], [16].

The proposed system uses flow-based network traffic features from CICIDS2017 and develops both binary and multiclass classification models. The binary model detects whether a traffic record is benign or malicious, while the multiclass model identifies the specific attack category. Similar to many existing ML-based IDS studies, the project evaluates models using accuracy, weighted F1-score, macro F1-score, ROC-AUC, confusion matrices, and per-class F1-score [4], [5]. However, the project also considers data leakage, feature stability, class imbalance, and time-based evaluation to avoid relying only on optimistic performance results [6], [7], [10].

Compared with deep learning and transformer-based IDS systems, the proposed system uses Random Forest as the final deployed model because it provides strong performance, stable behavior, lower deployment complexity, and compatibility with SHAP-based explanation. Transformer-based systems such as RTIDS can achieve strong results [12], but they may require higher computational resources and their future work still focuses on improving quick-response detection speed. For a graduation project prototype, a Random Forest-based structure is more practical for integration with FastAPI and a real-time-like dashboard [8].

Compared with federated learning-based IDS approaches, the main focus of the project is a centralized IDS architecture. However, an additional FedAvg-based federated learning simulation was implemented to investigate distributed training across multiple clients. Federated systems are important for IoT and multi-client environments because they reduce raw data transfer and preserve privacy [13], [14], [15]. However, they also introduce challenges such as non-IID data distribution, client participation, communication rounds, and aggregation stability. Our project focuses instead on a centralized prototype that demonstrates model deployment, alert visualization, and analyst support.

Compared with XAI-focused studies, the proposed system does not use explainability only as an offline analysis tool [16]. Instead, explainability is integrated into the dashboard. The backend generates feature-based explanation information and the frontend displays the most influential features for detected attacks. This makes the system more practical for analysts be-

cause the explanation is shown together with the prediction result, attack type, severity, and event timeline.

The main differences between existing systems and the proposed method are summarized in Table 2.1.

Table 2.1: Comparison Between Existing Systems and Proposed Method

Criterion	Existing Systems	Proposed Method
Detection approach	Signature-based, anomaly-based, ML, DL, or federated IDS approaches	ML-based binary and multi-class intrusion detection
Model focus	Often focuses mainly on offline model accuracy	Combines model training with API deployment and dashboard visualization
Dataset usage	Commonly uses benchmark datasets such as CICIDS2017, NSL-KDD, UNSW-NB15, TON-IoT, and CIC-DDoS2019	Uses CICIDS2017 with pre-processing, feature engineering, leakage analysis, and model evaluation
Class imbalance handling	Frequently discussed as a major challenge, especially for minority attack classes	Uses per-class F1-score and macro F1-score to evaluate minority class behavior
Leakage and dataset artifacts	Some benchmark datasets may include labeling errors, artifacts, and shortcut learning risks	Includes leakage-oriented analysis to remove features from the training feature set
Explainability	Often limited or used only as a separate analysis method	SHAP-based explanation is integrated into the analyst dashboard
Real-time usage	Many studies remain notebook-based or offline	FastAPI backend and React/Next.js dashboard provide real-time-like monitoring
Analyst support	Usually limited to model-level predictions and metrics	Provides traffic flow, attack distribution, alert timeline, severity, and key information
Deployment scope	Often model-only (notebook-based)	End-to-end prototype: model + API + dashboard + explainability

Overall, existing studies demonstrate that machine learning and deep learning techniques can be effective for intrusion detection [1], [2], [3]. However, they also reveal important limitations related to dataset quality, class imbalance, data leakage, interpretability, generalization, and deployment [6], [7], [10], [18]. The proposed project addresses these gaps by developing a practical IDS prototype that not only detects attacks but also presents the results in an analyst-oriented dashboard with explainability support [8], [9], [16].

3. METHODOLOGY

This chapter explains the methodology followed during the development of the AI-powered real-time network attack detection system and analyst dashboard. The project consists of two main parts: the machine learning pipeline and the software implementation. The machine learning pipeline includes dataset preparation, data cleaning, feature analysis, feature engineering, leakage analysis, model training, and model evaluation. The software implementation includes the development of a backend API, integration of trained models, and design of an analyst dashboard for monitoring prediction results.

The methodology was designed to create a complete intrusion detection prototype rather than only a standalone machine learning model. Therefore, the project workflow covers both experimental model development and practical system integration. The trained models are exported and served through a backend API, while the dashboard visualizes traffic statistics, prediction results, attack distributions, alert records, and explainability information.

The overall methodology of the project can be summarized as follows:



Figure 3.1: General Methodology Workflow

3.1 Needs Analysis

The first step of the project was to identify the functional and non-functional needs of the proposed intrusion detection system. Since the project aims to combine machine learning-based attack detection with an analyst dashboard, the system requirements were determined from both

data science and software engineering perspectives.

From the machine learning perspective, the system needs a reliable data processing pipeline that can prepare network traffic records for classification. This includes cleaning missing or invalid values, analyzing class distribution, selecting useful features, identifying misleading features, and training models for both binary and multiclass classification tasks. The system should be able to distinguish benign traffic from malicious traffic and classify the attack type when a malicious record is detected.

From the software perspective, the system needs a backend service that can load trained machine learning models and provide prediction-related data to the frontend. The backend should be able to return recent traffic events, historical prediction records, statistical summaries, and explanation data. The frontend dashboard should present this information in a clear and usable way for security analysts.

The analyst dashboard should allow users to monitor the current security status of the network traffic, inspect detected attacks, observe attack distributions, and review important details about recent alerts. In addition, explainability information should be provided to help analysts understand which features contributed to a model prediction.

The main requirements of the proposed system are summarized in Table 3.1.

Table 3.1: Functional and Non-Functional Requirements

Requirement Type	Requirement	Description
Functional	Binary traffic classification	The system must classify network traffic as benign or attack
Functional	Multiclass attack classification	The system must identify the specific attack category when malicious traffic is detected
Functional	Data preprocessing	The pipeline must handle missing values, duplicates, inconsistent labels, and unnecessary columns
Functional	Feature analysis	The system must identify and handle redundant, correlated, or problematic features
Functional	Model comparison	Multiple models must be trained and compared using classification metrics
Functional	Traffic monitoring	The dashboard must display live traffic events and recent security alerts
Functional	Attack distribution	The system must visualize the proportion of different detected attack types
Functional	Backend API service	The API must serve prediction, history, and statistical information to the frontend
Functional	Analyst dashboard	The dashboard must display live events, traffic statistics, and alert details
Functional	Explainability support	The system must provide SHAP-based feature explanations for detected attacks
Non-Functional	Latency	The API should respond quickly enough to support real-time-like monitoring
Non-Functional	Reliability	The system should load and serve trained models consistently without errors
Non-Functional	Maintainability	The codebase should be structured so that individual components can be updated independently
Non-Functional	Scalability	The architecture should allow future extensions such as database storage and live packet capture
Non-Functional	Extensibility	The design should support future integrations such as SIEM connection and authentication

3.2 Dataset Overview

The dataset used in this project is the CICIDS2017 dataset, a widely used benchmark dataset developed by the Canadian Institute for Cybersecurity (CIC) for network intrusion detection research [4], [7], [10]. It consists of network traffic records representing both benign and malicious network activities. Each record corresponds to a network flow and contains statistical features extracted from packet-level communication. These features describe various characteristics of network behavior, including flow duration, packet length statistics, byte counts, packet counts, inter-arrival times, protocol-related information, and other flow-based traffic measurements.

The dataset contains both benign and malicious traffic samples. Benign records represent normal network behavior, while malicious records represent different types of cyberattacks. Therefore, the dataset is suitable for both binary and multiclass classification tasks. In the binary classification task, the target variable is transformed into two categories: benign and attack. In the multiclass classification task, the model aims to identify the specific attack type.

A major characteristic of intrusion detection datasets is class imbalance. In real network traffic, benign records are usually more frequent than attack records, and some attack types may have significantly fewer samples than others. This imbalance can negatively affect model training because the model may become biased toward majority classes. Therefore, the class distribution of the dataset was analyzed before model training.

The initial class distribution of the dataset is shown in Figure 3.2. This analysis helped identify the imbalance between benign traffic and different attack categories. After observing the distribution, a controlled sampling strategy was applied to reduce the dominance of majority classes and prevent large daily traffic files from overwhelming the training data. No synthetic oversampling method such as SMOTE or random oversampling was used. Instead, the dataset was sampled using predefined caps: at most 50,000 benign records were selected from each daily traffic file, and at most 30,000 records were selected from each attack class when available.

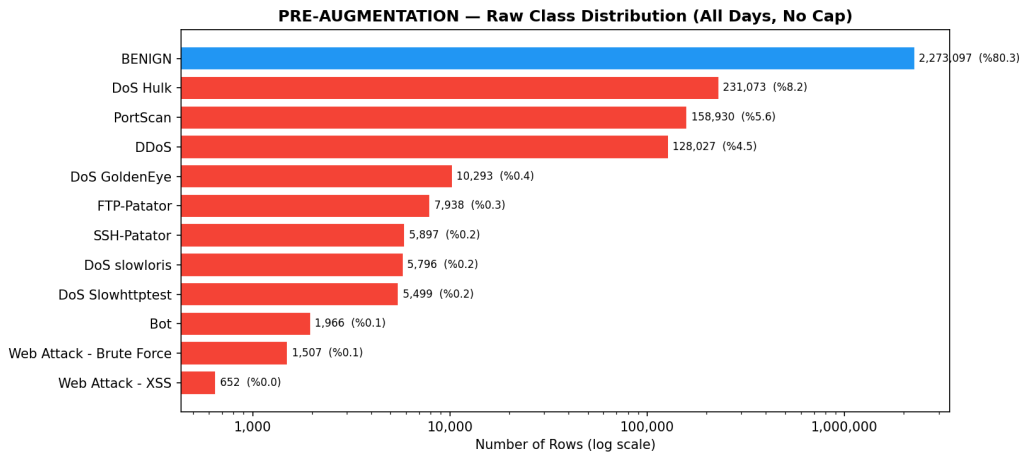


Figure 3.2: Class Distribution Before Controlled Sampling

To evaluate the effect of this controlled sampling strategy, the class distribution before and

after sampling was compared. As shown in Figure 3.3, the sampled dataset provides a more balanced and manageable representation of the selected classes without generating synthetic samples. This step was important for reducing the dominance of very large benign and attack classes while preserving real traffic records from the original dataset.

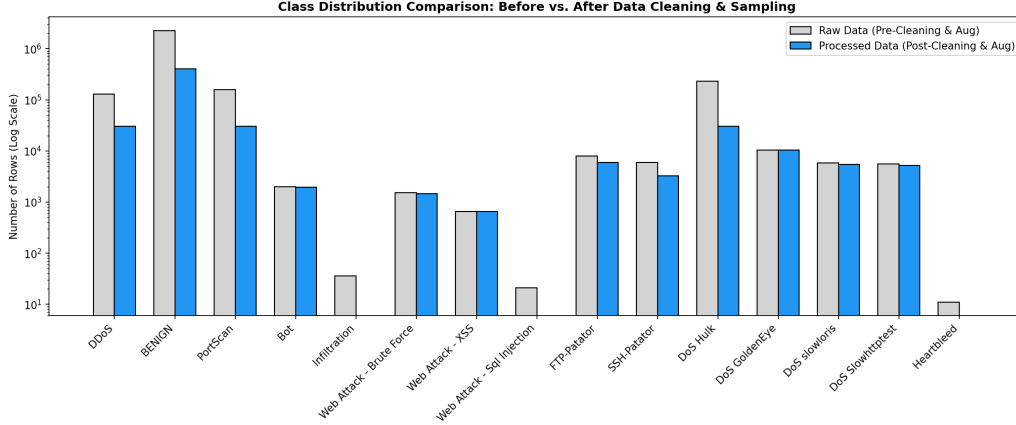


Figure 3.3: Class Distribution Before and After Controlled Sampling

The dataset was used as the main source for all machine learning experiments in this project. After the dataset overview stage, data cleaning, preprocessing, feature engineering, leakage analysis, model training, and model evaluation steps were performed systematically.

3.3 Data Preprocessing and Cleaning

Data preprocessing and cleaning is an important step in machine learning-based intrusion detection because raw network traffic datasets may contain missing values, infinite values, duplicate records, inconsistent labels, and unnecessary columns. If these issues are not handled properly, the trained model may learn incorrect patterns or produce unreliable results. Therefore, before model training, the dataset was cleaned and transformed into a suitable format for classification.

The first step of preprocessing was to inspect the dataset structure and identify problematic values. Missing values and infinite values were checked because some flow-based features may be generated through mathematical calculations such as ratios, rates, or time differences. These operations can produce undefined or infinite values when the denominator is zero or when traffic duration is extremely small. Such values were handled before training to prevent errors during model fitting.

Duplicate records were also examined and removed when necessary. Duplicate samples can cause the model to memorize repeated patterns and may lead to unrealistic evaluation results. Removing duplicates helps improve the reliability of the training and testing process.

The label column was then processed according to the classification task. For binary classification, labels were converted into two main categories: benign and attack. For multiclass classification, the original attack categories were preserved so that the model could learn to distinguish between different attack types. This allowed the project to evaluate both general attack

detection performance and detailed attack classification performance.

Unnecessary columns and non-informative values were also reviewed during preprocessing. Some columns may not contribute useful information to the learning process or may create technical problems during model training. Therefore, the dataset was prepared by keeping the relevant numeric traffic features and removing unsuitable fields when required.

After the cleaning stage, a controlled sampling strategy was applied to create a more balanced and manageable dataset. No synthetic oversampling technique such as SMOTE or random oversampling was used. Instead, the dataset was sampled using predefined limits based on daily traffic files and attack classes. A maximum of 50,000 benign records were selected from each daily traffic file, while a maximum of 30,000 records were selected from each attack class when available. This approach helped reduce the dominance of very large classes while preserving only real traffic records from the original dataset.

The sampling process was applied after cleaning to ensure that missing values, infinite values, and duplicate records did not affect the final dataset composition. As a result, the balancing process relied entirely on controlled selection of existing samples rather than generating synthetic data.

After the cleaning and sampling process, the class distribution of the resulting dataset was analyzed again. This step was performed to confirm that the preprocessing operations did not create unexpected changes in the dataset structure. The class distribution after cleaning is shown in Figure 3.4.

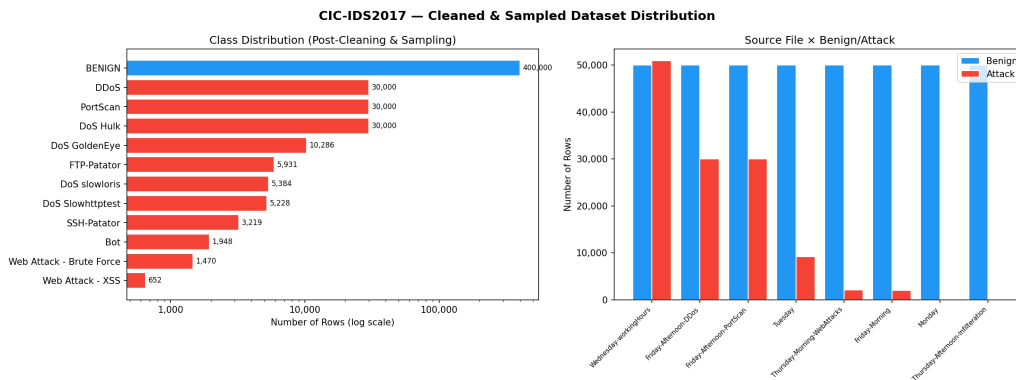


Figure 3.4: Class Distribution After Cleaning

The cleaned dataset was then used as the input for feature engineering, data leakage analysis, model training, and evaluation. By completing the preprocessing, cleaning, and sampling stages before feature selection and model development, the project aimed to improve the consistency, reliability, and reproducibility of the machine learning pipeline.

3.4 Feature Engineering

Feature engineering was performed to improve the quality of the input variables used by the machine learning models. In intrusion detection datasets, extracted flow-based features may

include redundant, highly correlated, sparse, skewed, or potentially unstable variables. If these features are used without analysis, the model may learn misleading patterns or become overly dependent on a small number of features. Therefore, feature engineering was applied after data preprocessing and before model training.

The first step of feature engineering was to examine the general behavior of the available features. Features with a high ratio of zero values were analyzed because some traffic attributes may remain inactive for most records. In network flow datasets, zero-heavy features are not always useless; they may represent meaningful behavior for specific traffic types. However, if a feature contains zero values for almost all records, it may provide limited information to the model. Therefore, such features were reviewed before the final model development stage. The zero-ratio analysis is shown in Figure 3.5.

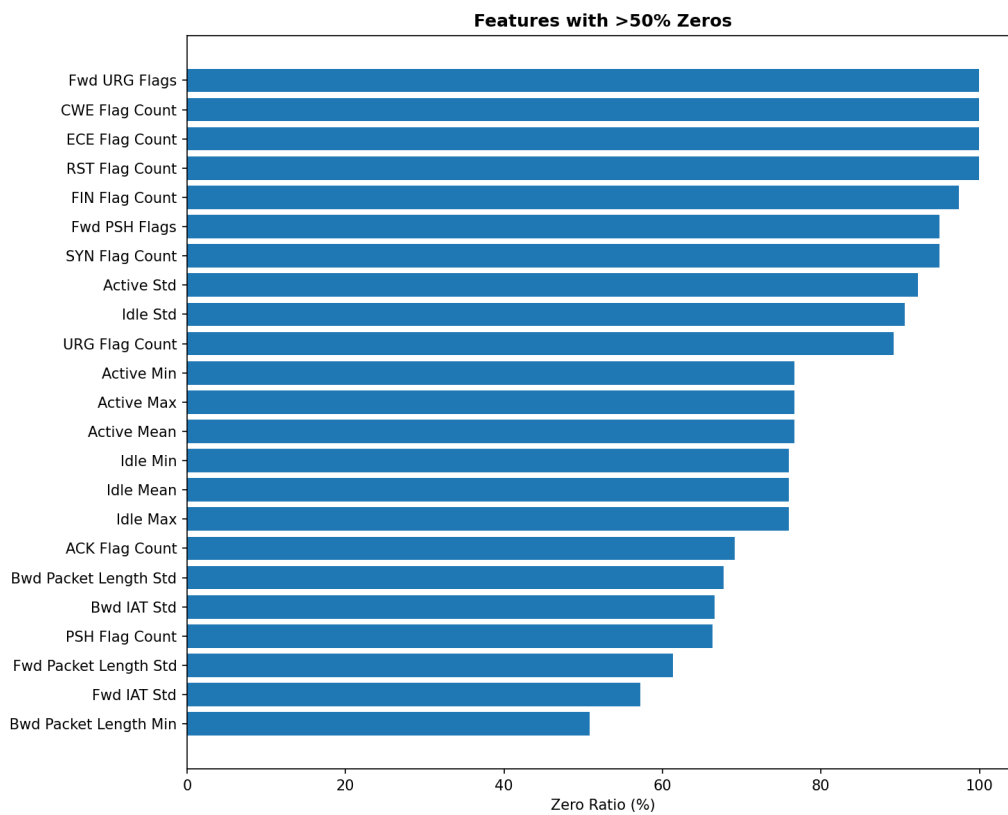


Figure 3.5: Features with High Zero Ratio

After examining zero-heavy features, mutual information analysis was used to measure the relationship between input features and the target label. Mutual information is useful because it can capture both linear and non-linear dependencies between features and class labels. Features with higher mutual information values are generally more informative for classification. In this project, mutual information scores were used as a guide to understand which features contributed more strongly to attack detection and classification. The mutual information-based feature importance results are presented in Figure 3.6.

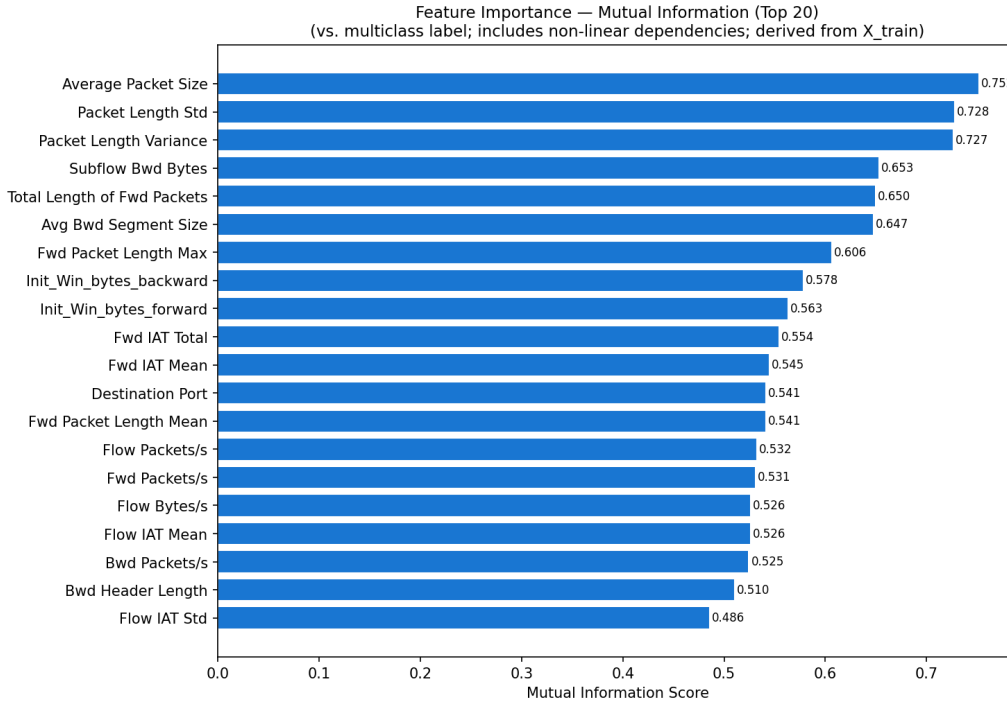


Figure 3.6: Mutual Information-Based Feature Importance

Skewness analysis was also performed to identify features with highly asymmetric distributions. Network traffic features often have skewed distributions because most records represent normal or repetitive traffic patterns, while a smaller number of records may contain extreme values caused by attack behavior or unusual network activity. Although skewed features can sometimes be useful for detecting anomalies, extremely skewed variables may also indicate unstable or misleading feature behavior.

During this analysis, the feature `min_seg_size_forward` was observed as a potentially problematic variable because it showed an extreme skewness value. Such a distribution may indicate that the feature is dominated by unusual values or dataset-specific artifacts. If the model depends heavily on this type of feature, the evaluation results may become less reliable and the model may not generalize well to unseen traffic. Therefore, the effect of this feature was examined carefully during the feature engineering process. The skewness analysis of selected features is shown in Figure 3.7.

In addition to `min_seg_size_forward`, the feature `Bwd Header Length` was also identified as a potentially problematic variable during the final model preparation stage. Both features exhibited highly irregular distribution characteristics and were considered susceptible to dataset-specific artifacts rather than representing stable network behavior. Therefore, these features were excluded from the final exported feature set used by the deployed models.

The feature engineering process was performed in multiple stages. The original dataset contained approximately 70 raw traffic features. After preprocessing, feature selection, and removal of non-informative variables, the feature set was reduced substantially. Additional filtering steps, including leakage-oriented analysis and skewness-based feature removal, resulted

in a final feature set consisting of 45 features. These 45 features were used for training and exporting the final machine learning models integrated into the backend system.

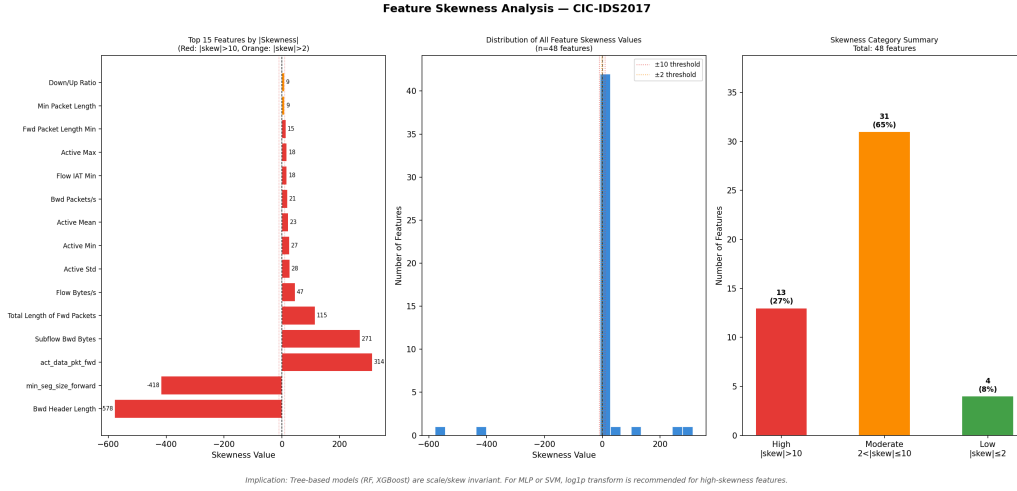


Figure 3.7: Skewness Analysis of Features

The final part of feature engineering was to evaluate how the feature processing decisions affected model performance. The purpose was not only to increase the performance scores, but also to create a more reliable and interpretable feature set. For this reason, model performance before and after feature engineering was compared. This comparison helped determine whether the feature engineering process improved the model or introduced any negative effect. The comparison of model performance before and after feature engineering is shown in Figure 3.8.

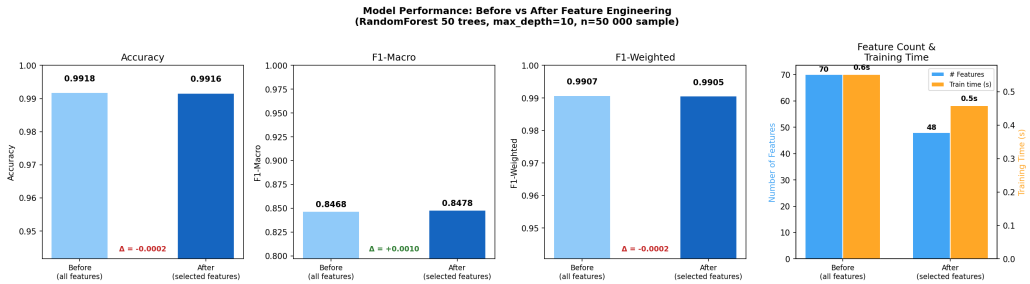


Figure 3.8: Model Performance Before and After Feature Engineering

Overall, the feature engineering stage supported the development of a more controlled machine learning pipeline. By analyzing zero-heavy features, informative features, skewed variables, and performance changes, the project reduced unnecessary feature noise and improved the reliability of the model development process. The outputs of this stage were then used in data leakage analysis, model training, and final model evaluation.

3.5 Data Leakage Analysis

Data leakage analysis was performed to evaluate whether the model could learn unrealistic or dataset-specific patterns instead of general network attack behavior. In machine learning-based

intrusion detection systems, data leakage is an important problem because some features may indirectly reveal the target class or may contain information that would not be available in a real-world detection scenario. If such features are used during training, the model may achieve very high evaluation scores but fail to generalize to unseen network traffic.

In intrusion detection datasets, leakage may occur in different forms. Some features can be strongly associated with specific attack classes because of the way the dataset was collected or generated. For example, service-related variables, address-based information, timestamps, or flow identifiers may allow the model to memorize dataset-specific patterns rather than learn meaningful attack behavior. This situation is especially risky when the dataset contains repeated traffic structures or when specific attacks consistently appear under similar network conditions.

For this reason, leakage-sensitive feature groups were examined carefully before the final model export stage. The primary focus of the analysis was the Destination Port feature. Although destination port information can be useful in real-world network security monitoring, it may also reflect dataset-specific service configurations and attack generation procedures. As a result, a model may learn shortcut relationships between port numbers and attack labels rather than general attack characteristics. To reduce this risk, Destination Port was removed from the final training feature set after the leakage analysis process.

The selected feature set was also reviewed for explicit flow identifier-like variables such as Flow ID, source IP address, destination IP address, and timestamp-related attributes. These variables are frequently reported as potential sources of shortcut learning in intrusion detection research. However, such explicit identifier fields were not present in the final selected feature set used for model training. Therefore, no additional identifier-based feature removal was required beyond the leakage review process.

In addition to leakage-sensitive variables, features exhibiting highly irregular statistical behavior were also reviewed because they could encourage the model to rely on unstable dataset-specific artifacts. The features `min_seg_size_forward` and `Bwd Header Length` showed highly unusual distribution characteristics and were therefore excluded from the final exported feature set. Although these variables are not direct leakage features, removing them helped create a more stable and generalizable feature representation.

The leakage-oriented feature analysis is summarized in Table 3.2.

Table 3.2: Leakage-Oriented Feature Analysis

Feature / Feature Group	Reason for Analysis	Possible Risk	Action Taken
Destination Port	Some attack types may be associated with specific service ports in the dataset	The model may learn port-specific patterns instead of behavioral ones	Evaluated during leakage analysis and excluded from the final training feature set
Address-based features	Source or destination address-related fields were checked for possible dataset-generation patterns	Explicit address-based identifiers were not present in the selected final feature list	Reviewed during leakage analysis; no explicit address-based feature was retained in the final feature set
Flow identifier-like fields	The selected dataset version was checked for fields such as Flow ID, source IP, destination IP, and timestamp	Such fields could cause memorization if present in the training feature set	No explicit flow identifier-like field was present in the final selected feature list
Highly correlated features	Some variables may carry redundant or equivalent information	The model may rely on duplicated signals	Reviewed during feature analysis and engineering
min_seg_size_forward	The feature showed an extreme skewness value	The model may rely on an unstable or misleading distribution	Excluded from the final exported feature set due to its highly unstable distribution
BwdHeaderLength	The feature also showed highly irregular distributional behavior during final model preparation	The model may learn dataset-specific artifacts instead of stable traffic behavior	Excluded from the final exported feature set together with min_seg_size_forward

After analyzing leakage-sensitive and unstable features, the model development process focused on using a more reliable feature set. The objective of this stage was not only to maximize performance metrics but also to reduce the possibility of misleading results caused by dataset-specific artifacts and shortcut learning. In this context, a small decrease in performance after removing leakage-sensitive features should be interpreted as a more realistic evaluation outcome rather than a reduction in model quality.

Overall, the data leakage analysis improved the reliability of the project by making the evaluation process more representative of real-world intrusion detection scenarios. As a result of

preprocessing, feature engineering, leakage-oriented filtering, and final feature selection, the deployed machine learning models were trained using a final feature set consisting of 45 features.

3.6 Model Development

Model development was performed after data preprocessing, feature engineering, and data leakage analysis. The main goal of this stage was to train machine learning models that can detect malicious network traffic and classify attack types based on flow-based network traffic features. Since the project aims to build both a detection system and an analyst dashboard, the model development process was designed to support both prediction accuracy and practical system integration.

Two classification tasks were considered in this project. The first task was binary classification, where each traffic record was classified as either benign or attack. This task represents the main detection problem of the intrusion detection system. The second task was multiclass classification, where malicious traffic records were classified according to their specific attack categories. This task provides more detailed information to the analyst by identifying the type of detected attack.

Several machine learning algorithms were evaluated during model development. Tree-based ensemble models were preferred because they generally perform well on tabular network traffic datasets and can handle complex relationships between features. In this project, models such as Random Forest, XGBoost, and CatBoost were compared using common classification metrics. These models were selected because they are widely used in structured data classification problems and can capture non-linear patterns in network traffic behavior.

Random Forest was used as one of the main models in the project because of its robustness, interpretability, and strong performance on tabular datasets. It combines multiple decision trees and reduces the risk of overfitting compared to a single decision tree. XGBoost and CatBoost were also evaluated as gradient boosting-based alternatives. These models are designed to improve classification performance by combining weak learners in a sequential manner.

The model training process was conducted separately for binary and multiclass classification. For binary classification, the target labels were converted into benign and attack classes. For multiclass classification, the original attack categories were preserved to allow the model to learn class-specific patterns. The same preprocessing and feature engineering principles were applied consistently to ensure that the models were trained on reliable and comparable feature sets.

After training, the models were evaluated using standard classification metrics such as accuracy, precision, recall, and F1-score. Since intrusion detection datasets may include class imbalance, F1-score and per-class performance were considered especially important. Accuracy alone may not be sufficient because a model can achieve a high accuracy score by correctly predicting the majority class while performing poorly on minority attack classes.

The comparison of the trained models is shown in Figure 3.9. This comparison was used to select the most suitable model for the final system integration.

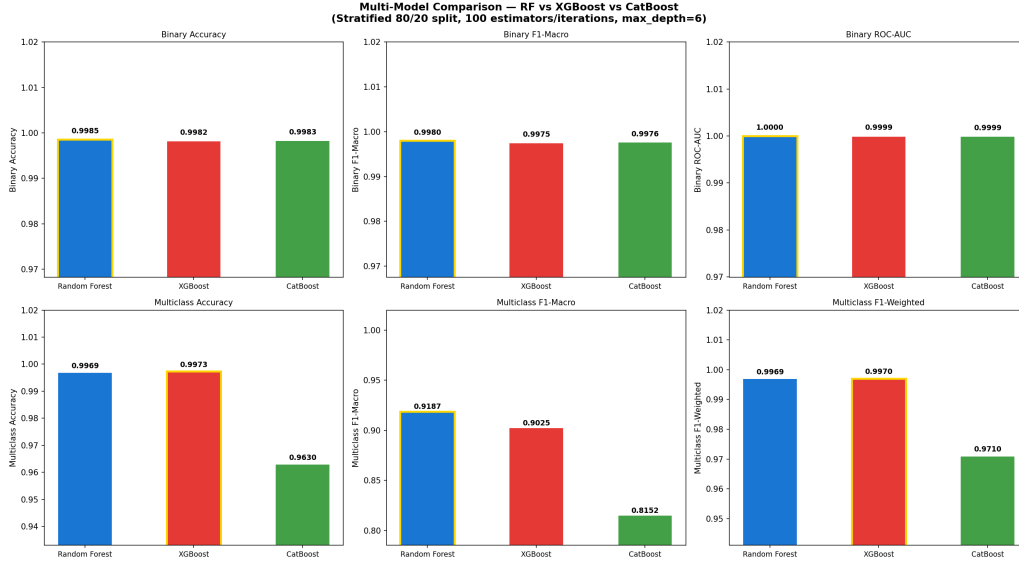


Figure 3.9: Model Comparison Overview

After the evaluation stage, the selected binary and multiclass models were exported and saved as model files. These saved models were later loaded by the backend API and used to generate predictions for the analyst dashboard. This export step was important because it connected the experimental machine learning pipeline with the software implementation part of the project.

Overall, the model development stage provided the core intelligence of the proposed intrusion detection system. By developing both binary and multiclass classifiers, the system was able to detect whether a traffic record was malicious and provide additional information about the possible attack type.

3.7 Model Evaluation and Cross-Validation

Model evaluation was performed to measure the performance and reliability of the trained machine learning models. Since the project includes both binary and multiclass classification tasks, the evaluation process was designed to analyze the models from different perspectives. The binary classification model was evaluated according to its ability to distinguish benign traffic from malicious traffic, while the multiclass classification model was evaluated according to its ability to identify specific attack categories.

Before final evaluation, the dataset was divided into training and testing subsets. The training set was used to teach the model the relationship between network traffic features and class labels, while the testing set was used to evaluate the model on unseen samples. This separation was important to reduce the risk of evaluating the model on the same data used for training.

In addition to the train-test evaluation, cross-validation was used to obtain a more reliable es-

timate of model performance. Cross-validation divides the dataset into multiple folds and trains the model repeatedly using different training and validation splits. This approach helps reduce the dependency of the results on a single train-test split and provides a more stable evaluation of the model.

In this project, both standard 5-fold cross-validation and stratified 5-fold cross-validation were applied using the Random Forest model. Stratified cross-validation was used to preserve class proportions across folds, which is particularly important for intrusion detection datasets that contain imbalanced attack categories. The evaluation was performed for both binary and multiclass classification tasks using accuracy, weighted F1-score, and macro F1-score. Comparing standard and stratified cross-validation results helped verify that the observed performance was not dependent on a specific data partition and provided additional confidence in the stability of the selected model.

Several evaluation metrics were used in this project. Accuracy was used to measure the overall ratio of correct predictions. However, accuracy alone is not sufficient for intrusion detection datasets because class imbalance may cause misleading results. For example, if benign traffic is much more frequent than attack traffic, a model may achieve high accuracy while performing poorly on minority attack classes. Therefore, precision, recall, and F1-score were also used.

Precision measures how many records predicted as attacks are actually attacks. Recall measures how many actual attack records are correctly detected by the model. F1-score combines precision and recall into a single metric and is especially useful when there is an imbalance between classes. For multiclass classification, per-class precision, recall, and F1-score were examined to understand how well the model performs on each attack category.

The evaluation metrics used in this project are summarized in Table 3.3.

Table 3.3: Evaluation Metrics Used in the Project

Metric	Description	Importance for IDS
Accuracy	Measures the overall ratio of correct predictions	Useful for general performance overview, but may be misleading when the dataset is imbalanced
Precision	Measures how many records predicted as attacks are actually attacks	Helps reduce false alarms by minimizing unnecessary analyst workload
Recall	Measures how many actual attack records are correctly detected by the model	Important for minimizing missed attacks
F1-score	Harmonic mean of precision and recall	Provides balanced performance measure, especially for imbalanced datasets
Confusion matrix	Shows correct and incorrect predictions for each class	Helps identify which classes are confused by the model
Per-class F1-score	Shows F1-score separately for each class	Important for evaluating minority attack classes in multiclass classification
Cross-validation score	Measures model performance across multiple data splits	Provides a more stable estimate than a single train-test split

Confusion matrices were used to analyze prediction errors in more detail. For binary classification, the confusion matrix shows the number of benign and attack records that were correctly or incorrectly classified. For multiclass classification, the confusion matrix shows which attack categories were confused with each other. This analysis is important because two models with similar accuracy values may behave differently for specific attack types.

Overall, the model evaluation and cross-validation stage ensured that the selected model was assessed not only by overall accuracy but also by more meaningful metrics for intrusion detection. This helped provide a more reliable understanding of the model's strengths and limitations.

3.8 System Architecture

The proposed system architecture was designed to connect the machine learning models with a backend API and an analyst dashboard. The main purpose of the architecture is to transform the trained intrusion detection models into a usable monitoring prototype. Therefore, the system includes three main layers: the machine learning layer, the backend service layer, and the

frontend dashboard layer.

The machine learning layer is responsible for preparing the dataset, training the classification models, and exporting the final models. In this project, separate models were developed for binary and multiclass classification. The binary model is used to detect whether a network traffic record is benign or malicious, while the multiclass model is used to classify the specific attack type when malicious traffic is detected. After training and evaluation, the selected models were saved as model files and used by the backend service.

The backend service layer was implemented to serve the trained models and provide prediction-related data to the frontend. The backend loads the exported machine learning models and generates prediction outputs for network traffic records. It also stores recent prediction events and provides statistical summaries for the dashboard. This layer acts as a bridge between the machine learning models and the user interface.

The frontend dashboard layer was designed for security analysts. It receives data from the backend API and visualizes the current state of the monitored traffic. The dashboard includes live prediction events, statistical cards, traffic charts, attack distribution charts, alert records, and explainability information. These components allow the analyst to observe both general traffic behavior and detailed attack-related information.

The system architecture of the proposed intrusion detection prototype is shown in Figure 3.10.

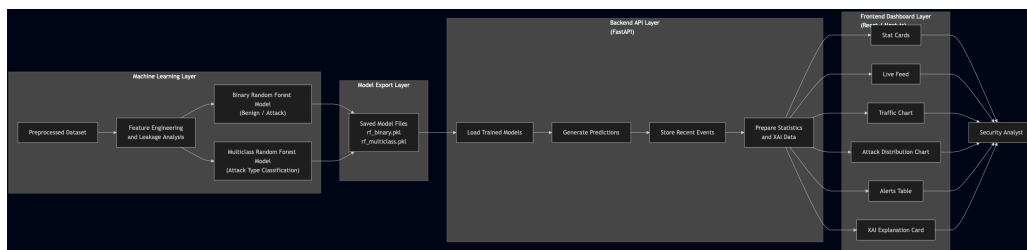


Figure 3.10: Proposed System Architecture

The data flow starts with preprocessed network traffic records. These records are used by the trained models to generate binary and multiclass predictions. The backend API processes the model outputs and prepares them as structured responses. The frontend dashboard then displays these responses through visual components. This structure makes the system modular because the machine learning models, backend service, and frontend interface can be updated independently.

The architecture also supports future improvements. For example, the current prototype can be extended with live packet capture, database storage, authentication, notification mechanisms, or SIEM integration. Since the model layer, backend layer, and frontend layer are separated, these improvements can be added without redesigning the entire system.

Overall, the system architecture provides an end-to-end structure for an AI-powered intrusion detection prototype. It demonstrates how trained machine learning models can be integrated

into a backend service and presented through an analyst-oriented dashboard.

3.9 Backend API Implementation

The backend API was developed to connect the trained machine learning models with the analyst dashboard. In this project, FastAPI was used as the backend framework because it provides a lightweight, fast, and flexible structure for building API-based machine learning services. The backend acts as the communication layer between the exported models and the frontend dashboard.

The backend loads the exported binary and multiclass Random Forest models from saved model files. The binary model is responsible for detecting whether a traffic record is benign or malicious, while the multiclass model is responsible for identifying the specific attack category. Each saved model file also includes the feature list used during training. This ensures that incoming traffic records are aligned with the same feature structure before prediction.

The prediction process is based on preprocessed holdout traffic records. When the stream endpoint is called, the backend selects a sample traffic record, prepares the input features according to the binary and multiclass model requirements, and generates predictions. The binary prediction determines whether the traffic is benign or attack. If the traffic is detected as an attack, the multiclass prediction and explainability information are used to provide more detailed information to the dashboard.

The backend also maintains a recent attack history. In the implemented prototype, detected attack events are stored in a limited in-memory queue. This structure allows the dashboard to retrieve recent alerts without requiring a separate database system. The history mechanism is suitable for prototype-level monitoring and can be extended with persistent database storage in future versions.

In addition to predictions and history, the backend provides dashboard-oriented summary and traffic statistics through dedicated endpoints. These statistics include the total number of processed events, attack and benign traffic counts, attack type distributions, and time-based traffic information used by dashboard visualizations. The frontend uses this information to update visual components such as statistical cards and attack distribution charts.

The main backend API endpoints are summarized in Table 3.4.

Table 3.4: Backend API Endpoints

Endpoint	Method	Purpose	Output
/	GET	Checks whether the IDS backend service is running	System status, version information and API availability
/predict/stream	GET	Selects a traffic sample, generates binary and multiclass predictions, and returns a stream event	Event ID, timestamp, binary prediction, multiclass prediction, and SHAP explanations when applicable
/history	GET	Returns recent detected attack events stored by the backend	Total number of historical events and recent alert records
/history/stats	GET	Provides attack statistics for dashboard visual components	Total detected attacks and attack type distribution
/dashboard/summary	GET	Returns dashboard-level summary statistics used by overview cards and monitoring widgets	Total events, attack count, benign count, recent activity statistics and dashboard summary information
/dashboard/traffic	GET	Provides time-based traffic information for dashboard visualizations	Traffic timeline data used by charts and monitoring components
/debug/shap	GET	Test-time SHAP explainer output during development	SHAP output type, shape and explanation feature information

The backend was also configured with Cross-Origin Resource Sharing support. This allows the frontend application to communicate with the API during development. Since the frontend and backend may run on different local ports, CORS configuration is necessary for smooth integration between the two components.

For explainability, the backend initializes SHAP TreeExplainer objects for the trained models. When an attack is detected, the backend calculates the most influential features for the prediction and returns them as part of the event response. Each explanation item includes the feature name, SHAP value, actual feature value, risk direction, description, and severity hint.

This information is later displayed in the dashboard through the XAI explanation component.

The backend implementation was designed as a prototype-level service. It does not use a permanent database in the current version; instead, recent events are stored temporarily in memory. This design keeps the system simple and suitable for demonstration. However, the architecture can be extended with database storage, authentication, live traffic ingestion, and deployment configurations in future work.

Overall, the backend API provides the operational bridge between the machine learning models and the analyst dashboard. It allows the trained models to be used outside the notebook environment and supports real-time-like monitoring through API-based prediction, history, statistics, and explainability responses.

3.10 Analyst Dashboard Design

The analyst dashboard was designed to visualize the outputs of the intrusion detection system in a clear and usable interface. While the machine learning models provide prediction results, these results need to be presented in a meaningful way for security analysts. Therefore, the dashboard was developed as the frontend layer of the project and was connected to the backend API.

The dashboard was implemented using React and Next.js. It receives prediction events, historical alert records, attack statistics, and explanation data from the FastAPI backend. The main purpose of the dashboard is to support real-time-like monitoring by displaying recent traffic events and detected attacks through visual and interactive components.

The dashboard includes several components designed for security monitoring. Statistical cards summarize the current state of the system, such as total traffic records, detected attacks, benign traffic, and critical alerts. These cards allow the analyst to quickly understand the overall security status without inspecting every record individually.

A live feed component displays the most recent prediction event returned by the backend. This component helps the analyst follow the latest traffic activity and observe whether the current record is classified as benign or malicious. In addition, traffic charts are used to visualize traffic behavior over time, while the attack distribution chart shows the proportion of different attack types detected by the system.

The alerts table is one of the main components of the dashboard. It lists detected attack events with information such as timestamp, attack type, severity level, confidence, and related details. This table allows the analyst to inspect recent attacks in a structured way. Filtering and expandable details can be used to make the alert review process easier and more efficient.

The dashboard also includes an explainability component. This component presents feature-based explanation information generated by the backend. Instead of only showing the final prediction, the dashboard provides additional information about the important features that influenced the model output. This makes the system more transparent and helps the analyst understand why a traffic record was classified as malicious.

The general dashboard interface is shown in Figure 3.11.

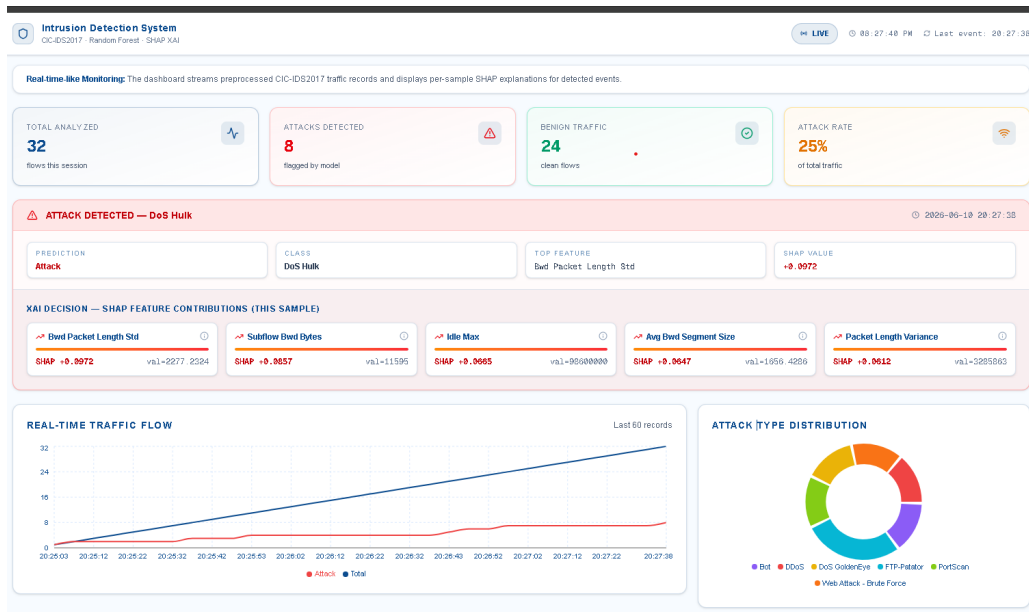


Figure 3.11: Analyst Dashboard Overview

An example of the alert details and explainability section is shown in Figure 3.12. This view demonstrates how the system provides additional context for detected attacks and supports analyst interpretation.

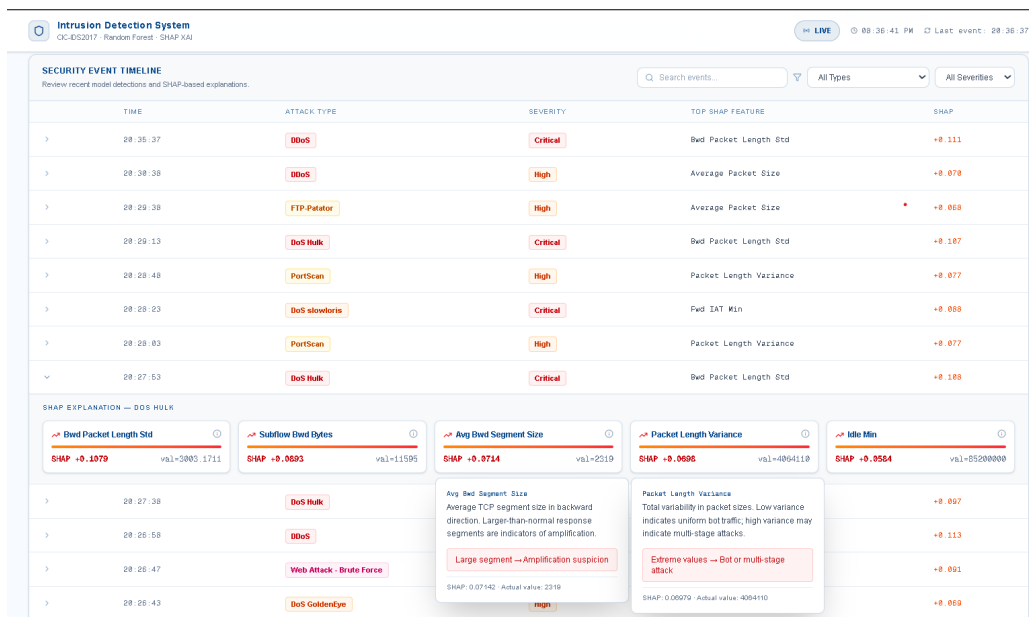


Figure 3.12: Alert Details and XAI Explanation

The dashboard design focuses on usability, readability, and fast interpretation. Since intrusion detection systems may generate many alerts, the interface was designed to reduce visual complexity and highlight important information. Color-coded indicators, statistical summaries, charts, and structured alert records help the analyst identify suspicious activity more easily.

Overall, the analyst dashboard transforms raw model outputs into actionable visual infor-

mation. It supports the practical use of the proposed intrusion detection prototype by combining prediction results, attack statistics, alert history, and explainability information in a single interface.

3.11 Explainability Module

The explainability module was developed to make the machine learning predictions more understandable for security analysts. In intrusion detection systems, it is not sufficient to only display whether a traffic record is benign or malicious. Analysts also need to understand why the model produced a specific prediction, especially when the system detects suspicious or critical traffic. For this reason, an explainability component was included in the proposed system.

Machine learning models, especially ensemble-based models, can capture complex relationships between features. However, this complexity may make their decisions difficult to interpret. If the system only provides the final prediction label, the analyst may not be able to understand which traffic characteristics contributed to the decision. This can reduce trust in the system and make alert investigation more difficult.

In this project, feature-based explanation information was generated for model predictions. The backend analyzes the contribution of input features and identifies the most influential factors behind a prediction. These explanation results are then sent to the frontend dashboard and displayed through the XAI explanation component.

The explainability module presents information such as the feature name, feature value, contribution direction, severity hint, and a short explanation. This allows the analyst to observe which traffic characteristics increased or decreased the risk score. For example, features related to packet length, flow duration, byte rate, or packet count may help explain why a specific traffic record was classified as malicious.

The XAI component was designed to support analyst interpretation rather than replace human decision-making. The purpose of the explanation output is to provide additional context for detected alerts. By reviewing the important features behind a prediction, the analyst can better understand whether the alert is consistent with suspicious network behavior.

The explainability information is integrated into the dashboard together with alert details, as shown in Figure 3.12. Overall, the explainability module improves the transparency of the proposed intrusion detection prototype. It helps transform the system from a black-box classifier into a more interpretable decision-support tool for network security monitoring.

3.12 Federated Learning Simulation

As an extension of the proposed intrusion detection system, a FedAvg-based federated learning simulation was implemented. The purpose of this experiment was to investigate whether a machine learning-based intrusion detection model can be trained across multiple distributed clients without directly combining their local training data.

In this simulation, the prepared CICIDS2017 feature dataset was divided into five client partitions. Each client trained a local neural network model on its own local data for two local epochs in each communication round. After local training, the clients sent their model parameters to a central server. The server then aggregated the received parameters using sample-weighted Federated Averaging (FedAvg). This process was repeated for five communication rounds.

The federated learning model used 45 selected input features and was evaluated on a separate holdout test set. In total, 419,294 samples were used for distributed training and 104,824 samples were kept for final holdout evaluation. Unlike the earlier client-based evaluation setup, this experiment included local client training, communication rounds, and global model aggregation.

The final FedAvg model achieved an accuracy of 0.9695, a weighted F1-score of 0.9672, and a macro F1-score of 0.7451 on the holdout test set. These results show that the federated learning setup was able to learn useful attack detection patterns without centralizing the training data. However, the macro F1-score was lower than the centralized Random Forest model, indicating that minority class detection remains more challenging in the federated setting. In this experiment, the federated model was implemented using a three-layer Multi-Layer Perceptron (MLP) neural network architecture rather than the centralized Random Forest model used in the deployed intrusion detection system. The simulation was performed using five clients, five communication rounds, and IID client partitioning. Therefore, the results demonstrate the feasibility of FedAvg-based distributed training; however, they do not fully represent the challenges of realistic non-IID federated environments where client data distributions may differ significantly. The final federated learning performance is summarized in Table 3.5.

Table 3.5: FedAvg Federated Learning Results

Metric	Value
Accuracy	0.9695
Weighted F1	0.9672
Macro F1	0.7451
Communication Rounds	5
Clients	5
Local Epochs	2

The federated learning simulation should be considered as an experimental extension of the centralized IDS architecture. While the deployed backend relies on Random Forest models for prediction, the federated component was designed to evaluate distributed neural network training without sharing raw client data.

3.13 Testing

Testing was performed to verify that the machine learning models, backend API, and frontend dashboard work correctly as an integrated intrusion detection prototype. Since the project consists of multiple components, the testing process was applied at different levels, including model testing, API testing, dashboard testing, and integration testing.

The first part of the testing process focused on the machine learning models. The binary classification model was tested to confirm that it could distinguish benign traffic from attack traffic. The multiclass classification model was tested to evaluate whether it could classify detected attacks into specific attack categories. Classification metrics, confusion matrices, and per-class performance values were used to analyze the model outputs. The detailed experimental results are presented in Chapter 4.

The backend API was tested to ensure that the exported models could be loaded correctly and used for prediction. The API endpoints were checked to verify that they returned properly structured responses for stream events, history records, statistics, and explainability information. This step was important because the dashboard depends on consistent backend responses to update its visual components.

The frontend dashboard was tested by checking whether the interface correctly displayed the data received from the backend. Statistical cards, live feed, traffic charts, attack distribution charts, alerts table, and XAI explanation components were reviewed during this process. The dashboard was also tested for readability and usability so that the presented information could be interpreted easily by a security analyst.

Integration testing was performed to verify the communication between the backend and frontend. During this test, the backend generated prediction events and the frontend consumed these events through API requests. The dashboard was expected to update its components according to the received data. This confirmed that the machine learning models, API layer, and dashboard interface were working together as a complete system.

The main testing activities are summarized in Table 3.6.

Table 3.6: System Testing Summary

Test Area	Test Case	Expected Result	Status
Model testing	Binary classification prediction	The model identifies traffic as benign or attack	Passed
Model testing	Multiclass classification prediction	The model correctly identifies attack category when malicious traffic is detected	Passed
Model testing	Cross-validation metric calculation	Accuracy, recall, F1-score and similar metrics are computed correctly	Passed
Backend API testing	Root endpoint check	The API returns system status and API availability information	Passed
Backend API testing	Stream endpoint response	The API successfully generates a prediction event	Passed
Backend API testing	History endpoint response	The API correctly returns recent detected attack events	Passed
Backend API testing	Statistics endpoint response	The API returns total detected attacks and attack type distribution correctly	Passed
Dashboard testing	Statistical cards display	The dashboard correctly displays live events	Passed
Dashboard testing	Charts and tables display	Traffic charts, attack distribution and alert timeline display correctly	Passed
Integration testing	Backend-frontend communication	The dashboard successfully receives and visualizes data from the backend	Passed

The testing process showed that the developed prototype can load trained models, generate predictions, serve data through API endpoints, and visualize the results on the analyst dashboard. Since the current version is a prototype, the tests were performed in a local development environment. Future versions can include more advanced testing methods such as automated unit tests, load testing, database persistence testing, live traffic testing, and deployment testing.

Overall, the testing stage confirmed that the main components of the proposed system work together successfully. This completed the methodology phase and prepared the project for experimental result analysis.

4. EXPERIMENTAL RESULTS

This chapter presents the experimental results obtained from the machine learning models and the developed intrusion detection prototype. The results are presented using evaluation metrics, confusion matrices, model comparison outputs, per-class performance analysis, and dashboard screenshots. The main purpose of this chapter is to show how well the proposed system detects malicious traffic, classifies attack types, and presents the results through the analyst dashboard.

The experimental evaluation was conducted for two classification tasks. The first task was binary classification, where the model classified each traffic record as either benign or attack. The second task was multiclass classification, where the model predicted the specific attack category for malicious traffic records. In addition to model performance, the dashboard output was also evaluated to demonstrate how prediction results are visualized in the developed system.

Since intrusion detection datasets may contain class imbalance, the evaluation was not based only on accuracy. Weighted F1-score, macro F1-score, ROC-AUC, confusion matrices, and per-class F1-score values were also considered. These metrics provide a more detailed understanding of the model's behavior, especially for minority attack classes.

4.1 Final Binary Classification Performance

The binary classification model was evaluated to measure its ability to distinguish benign traffic from malicious traffic. The model was tested under two evaluation scenarios: per-file split and time-based split. The per-file split uses a stratified division of the dataset, while the time-based split separates training and testing data according to the temporal order of the records.

The binary classification performance comparison is summarized in Table 4.1.

Table 4.1: Binary Classification Performance Comparison

Model	Split	Accuracy	Weighted F1	Macro F1	ROC-AUC
Random Forest	Per-file	0.9985	0.9986	0.9980	1.0000
Random Forest	Time-based	0.8540	0.8197	0.6800	0.8637
XGBoost	Per-file	0.9982	0.9982	0.9975	0.9999
CatBoost	Per-file	0.9982	0.9982	0.9976	0.9999

As shown in Table 4.1, the Random Forest model achieved high performance in the per-file

binary classification scenario, with an accuracy of 0.9985 and a weighted F1-score of 0.9986. XGBoost and CatBoost also produced similar results in the per-file setting. However, the time-based split produced lower scores, which indicates that temporal separation creates a more challenging and realistic evaluation scenario. This result shows that the model performs very strongly under stratified/per-file evaluation, while real-world-like temporal generalization remains more difficult.

The binary classification confusion matrix is shown in Figure 4.1.

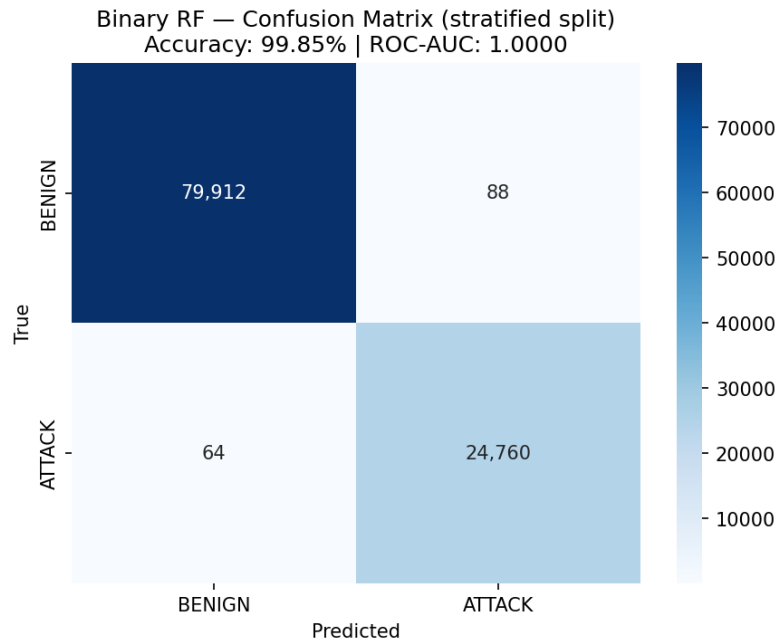


Figure 4.1: Binary Classification Confusion Matrix

4.2 Multiclass Classification Results

The multiclass classification model was developed to identify the specific type of attack after malicious traffic is detected. This task provides more detailed information for the analyst because different attack types may require different investigation and response strategies.

The multiclass model was evaluated using overall classification metrics and per-class performance values. Per-class evaluation is important because some attack categories may have fewer samples than others, and a model may perform well on majority classes while producing weaker results for minority classes.

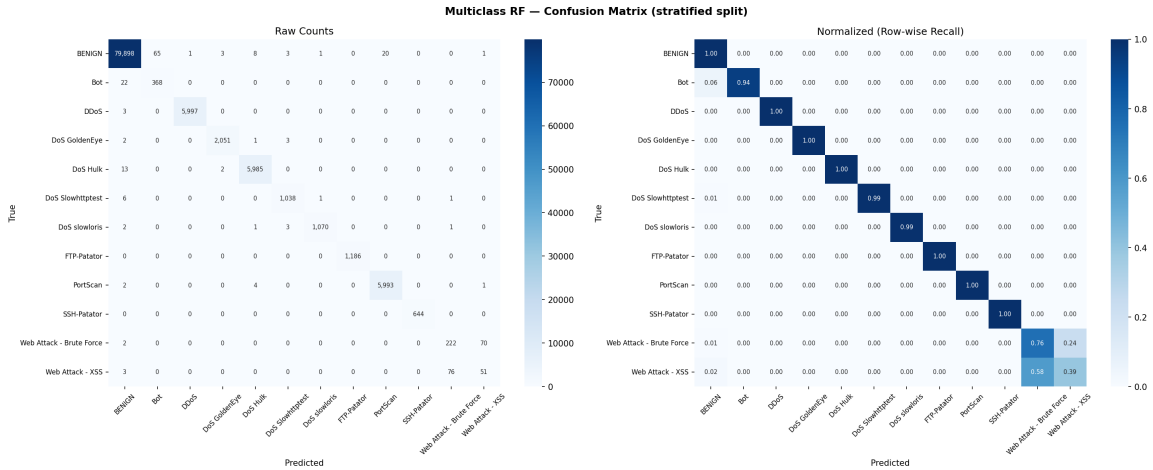
The final multiclass classification results are summarized in Table 4.2.

Table 4.2: Multiclass Classification Performance Comparison

Model	Split	Accuracy	Weighted F1	Macro F1
Random Forest	Per-file	0.9969	0.9969	0.9187
Random Forest	Time-based	0.7957	0.7119	0.0745
XGBoost	Per-file	0.9973	0.9970	0.9025
CatBoost	Per-file	0.9629	0.9710	0.8152

As shown in Table 4.2, Random Forest achieved an accuracy of 0.9969 and a weighted F1-score of 0.9969 in the per-file multiclass setting. Although XGBoost achieved a slightly higher accuracy score, Random Forest produced the highest macro F1-score among the evaluated multiclass models. This is important because macro F1-score gives equal importance to all classes and is more informative when the dataset contains minority attack categories. Therefore, Random Forest was selected as the final model for prototype integration.

The multiclass classification confusion matrix is shown in Figure 4.2.

**Figure 4.2:** Multiclass Classification Confusion Matrix

4.3 Model Comparison Results

Different machine learning models were compared to select the most suitable model for the proposed intrusion detection system. The comparison included tree-based ensemble models because they are effective for tabular datasets and can capture non-linear relationships between network traffic features.

The comparison focused on common classification metrics such as accuracy, weighted F1-score, macro F1-score, and ROC-AUC where applicable. However, the final model selection was not based only on the highest metric value. Stability, interpretability, deployment simplicity, and integration with the backend API were also considered.

The model comparison result is shown in Figure 4.3.

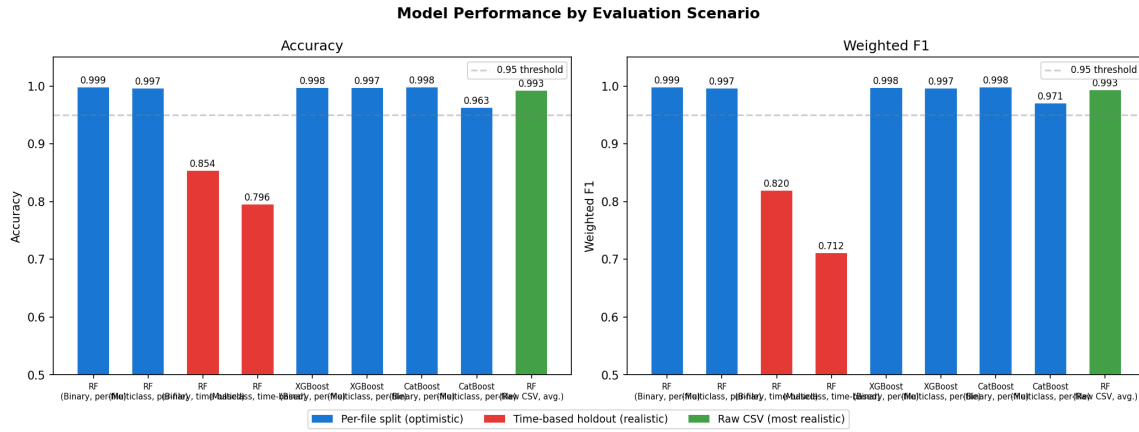


Figure 4.3: Model Performance by Evaluation Scenario

4.4 Per-Class F1-Score Analysis

Per-class F1-score analysis was performed to better understand the multiclass classification behavior of the model. In intrusion detection, overall accuracy may not be sufficient because some attack types may be underrepresented in the dataset. Therefore, per-class F1-score helps identify which attack categories are detected more reliably and which categories are more difficult for the model.

The per-class F1-score comparison is shown in Figure 4.4.

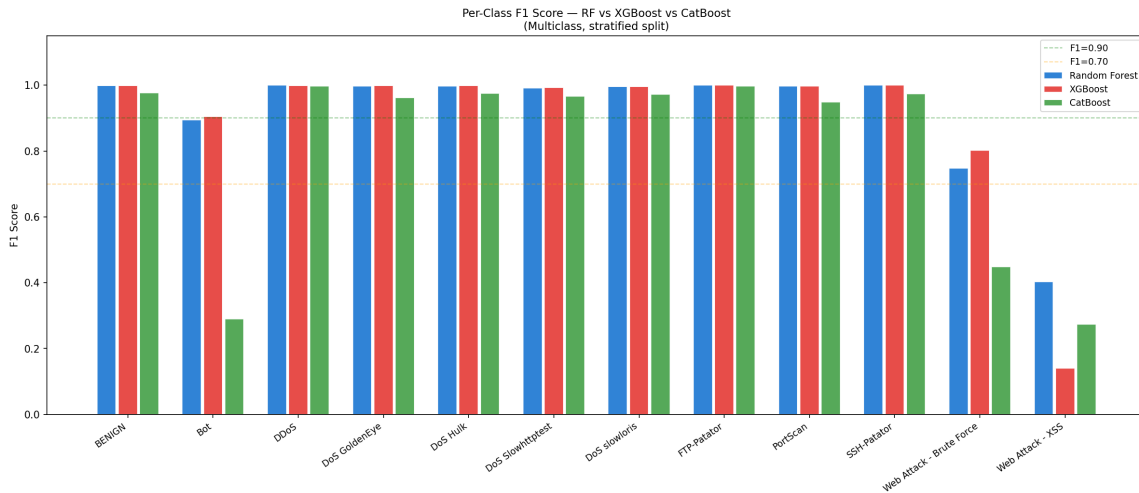


Figure 4.4: Per-Class F1-Score Comparison

Figure 4.4 shows that most major attack classes achieved high F1-scores across the evaluated models. However, lower F1-scores were observed for some minority web attack classes, especially Web Attack - XSS and Web Attack - Brute Force. This indicates that the model can classify frequent and well-represented attack categories more reliably, while minority and similar web attack classes remain more challenging.

4.5 Dashboard Output Results

In addition to model performance, the developed analyst dashboard was tested to verify that prediction results could be presented in a clear and usable interface. The dashboard receives data from the FastAPI backend and visualizes the current traffic status, detected attacks, attack distribution, recent alerts, and explainability information.

The dashboard output demonstrates that the trained models can be used outside the notebook environment and integrated into a working software prototype. This is important because the project aims to develop not only a machine learning model but also an end-to-end intrusion detection system.

Figure 4.5 shows the dashboard output during real-time-like monitoring. The visualization includes the current prediction panel, traffic flow chart, attack type distribution, and recent security event timeline. This output demonstrates that the prediction results generated by the backend can be transferred to the frontend and presented in an analyst-oriented format.

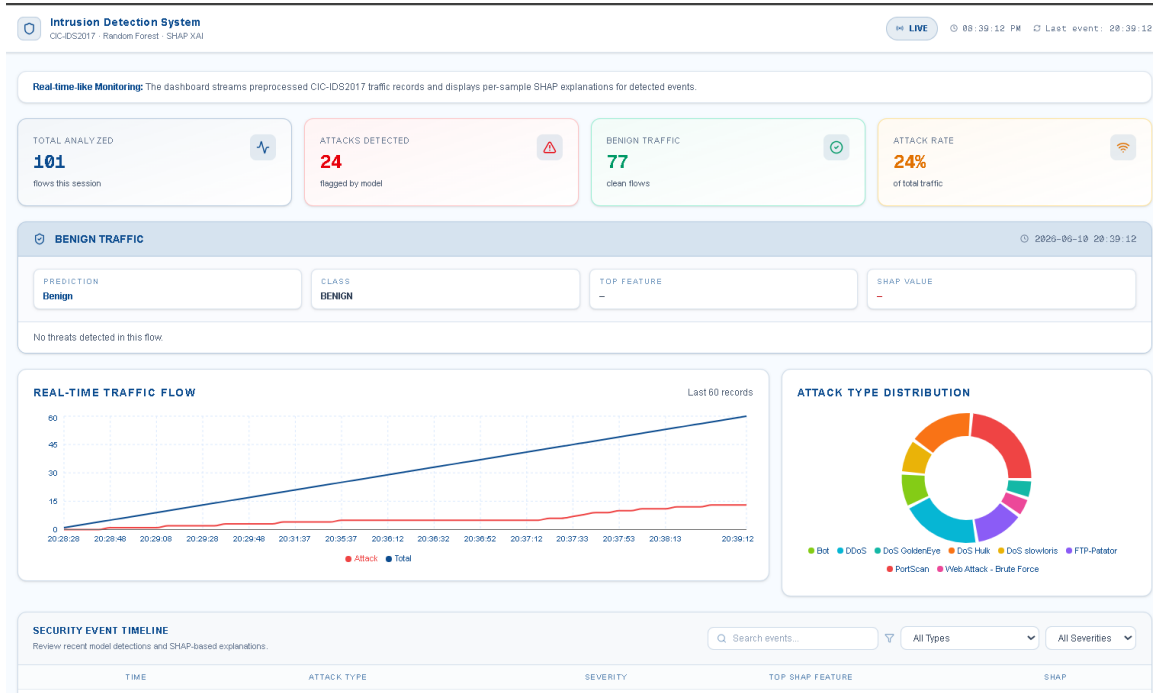


Figure 4.5: Dashboard Visualization of Detection Results During Real-Time-Like Monitoring

4.6 Summary of Experimental Findings

The experimental results show that the proposed system can perform both binary attack detection and multiclass attack classification. The binary model provides the main detection capability by identifying whether traffic is benign or malicious. The multiclass model extends this capability by predicting the specific attack type.

The evaluation results also show the importance of using multiple metrics instead of relying only on accuracy. Weighted F1-score, macro F1-score, ROC-AUC, confusion matrices, and

per-class F1-score values provide a more complete understanding of the model’s strengths and limitations. This is especially important for intrusion detection datasets because class imbalance can affect the reliability of the results.

The dashboard results demonstrate that the trained models were successfully integrated into a backend API and visualized through an analyst-oriented interface. Therefore, the experimental results support the main objective of the project: developing an AI-powered network attack detection prototype with real-time-like monitoring and explainability support.

4.7 Discussion of Results

The main problem addressed in this project is the need for an intrusion detection system that can automatically detect malicious network traffic and present the results in an understandable way for security analysts. Traditional intrusion detection approaches may be limited when attack behavior changes or when analysts need more contextual information about alerts. Therefore, this project focused on developing a machine learning-based detection model and integrating it with a backend API and an analyst dashboard.

The experimental results show that the proposed system can successfully perform both binary attack detection and multiclass attack classification. In the binary classification task, the model achieved strong results in the per-file evaluation scenario. This indicates that the model can effectively distinguish benign traffic from attack traffic when the training and test distributions are similar. The binary confusion matrix also shows that the number of false positives and false negatives is low in this setting.

However, the time-based evaluation results show that the detection problem becomes more difficult when the model is tested on temporally separated data [6], [7], [10]. This is an important finding because real-world network traffic changes over time. Attack behavior, traffic volume, and service usage patterns may differ between training and deployment periods. Therefore, the lower time-based performance should not be interpreted only as a weakness of the model; it also shows that temporal generalization is a realistic challenge for machine learning-based intrusion detection systems.

The contrast between the per-file and time-based evaluation results is particularly important. While the models achieved very strong performance under random and per-file evaluation strategies, the multiclass time-based Macro F1-score decreased substantially. This suggests that the learned attack patterns do not generalize equally well across different time periods. Therefore, the current model should not be interpreted as production-ready for long-term deployment without periodic retraining, drift monitoring, and validation on temporally separated data. The result highlights the importance of evaluating intrusion detection models under realistic temporal conditions rather than relying only on randomly shuffled datasets.

The extremely low multiclass time-based Macro F1-score should also be interpreted in the context of the chronological structure of the CICIDS2017 dataset. Some attack categories are concentrated on specific days and may be absent or significantly underrepresented in earlier

portions of the dataset. When earlier traffic periods are used for training and later periods are reserved for testing, the model may encounter attack classes that were rarely observed during training. As a result, reliable decision boundaries cannot be learned for those categories, leading to a substantial decrease in macro F1-score. Therefore, the time-based evaluation highlights not only the challenge of temporal generalization but also the effect of temporal class distribution shifts. This observation supports the argument that random or per-file evaluation strategies may produce overly optimistic intrusion detection results when temporal characteristics are ignored.

For multiclass classification, the results show that the model performs very well on most major attack categories. Classes such as DDoS, DoS Hulk, DoS GoldenEye, FTP-Patator, SSH-Patator, and PortScan achieved strong classification performance. This means that the model was able to learn distinctive traffic patterns for well-represented attack types. These results are important because identifying the attack category can help analysts prioritize incidents and select an appropriate response strategy.

On the other hand, lower performance was observed for minority web attack classes, especially Web Attack - XSS and Web Attack - Brute Force [3], [18]. This can be explained by the smaller number of samples, similarity between web attack behaviors, and possible overlap in their flow-based feature patterns. These results show that overall accuracy alone is not enough to evaluate an intrusion detection model. Macro F1-score and per-class F1-score are necessary to understand whether the model performs well across all attack categories.

The comparison between different models also provides important insights. Although XGBoost achieved slightly higher accuracy in some multiclass results, Random Forest produced a stronger macro F1-score and was selected for prototype integration. This decision was not based only on numerical performance. Random Forest was also preferred because of its stability, simpler deployment process, and compatibility with the explainability module used in the dashboard. In a practical prototype, model performance, interpretability, and integration simplicity should be considered together.

The cross-validation experiments further supported the stability of the Random Forest model. Both standard 5-fold cross-validation and stratified 5-fold cross-validation produced very similar results for binary and multiclass classification. This indicates that the selected model performs consistently across different random data partitions and that the reported performance is not strongly dependent on a specific train-test split. However, these findings should still be interpreted together with the time-based evaluation results, which provide a more realistic assessment of temporal generalization.

The feature engineering and leakage analysis stages also played an important role in improving the reliability of the results. In intrusion detection datasets, some features may create misleadingly high performance if they reflect dataset-specific patterns instead of general attack behavior. For this reason, leakage-sensitive features, highly skewed variables, and unstable features were examined carefully. The results showed that removing or controlling such features may slightly change the performance scores, but it helps create a more realistic and trustworthy

evaluation process.

Another important contribution of the project is the integration of the trained models into a working software prototype. Many machine learning studies report only offline model performance, but this project also demonstrates how the model outputs can be served through a backend API and visualized in a dashboard. The dashboard provides live prediction information, traffic flow visualization, attack type distribution, security event timeline, and SHAP-based explanation outputs. This makes the system more useful for analyst-oriented monitoring.

In addition to the centralized evaluation process, a FedAvg-based federated learning simulation was conducted using five distributed clients. Each client trained a local neural network model on its own data partition, and the central server aggregated the model parameters using sample-weighted Federated Averaging (FedAvg). The training process was repeated for five communication rounds with two local training epochs per round.

The final federated model achieved an accuracy of 0.9695, a weighted F1-score of 0.9672, and a macro F1-score of 0.7451 on the holdout test set. Although the centralized Random Forest model achieved higher multiclass performance, the federated approach demonstrated that effective attack detection can be achieved without centralizing training data. These results indicate that privacy-preserving distributed intrusion detection is feasible while maintaining strong overall classification performance. However, the lower macro F1-score suggests that minority attack classes remain more challenging in federated environments and may require additional optimization.

The explainability component is especially significant because intrusion detection alerts should be interpretable. If an analyst only sees a prediction label, it may be difficult to understand why the model generated that alert. By showing important features and their contribution values, the system provides additional context for decision-making [16]. This improves transparency and can increase trust in the model outputs.

There are also some limitations in the current version of the project. First, the real-time monitoring process is based on preprocessed traffic records rather than live packet capture from a real network interface. Therefore, the system should be considered a real-time-like prototype rather than a production-level live IDS. Second, the model was trained and evaluated on a specific dataset, so its performance may change when tested on different network environments. Third, the current backend stores recent events temporarily in memory and does not include persistent database storage. Finally, the dashboard does not yet include advanced features such as user authentication, notification mechanisms, or SIEM integration.

Possible sources of error include class imbalance, similar feature patterns between attack types, dataset-specific artifacts, and temporal distribution shifts. Minority attack classes may be more difficult to classify because they provide fewer examples during training. In addition, some attacks may produce similar network flow characteristics, which can cause confusion between classes. These issues are common in machine learning-based intrusion detection and should be considered when interpreting the results.

In the broader context, this project shows that machine learning models can be transformed into practical intrusion detection tools when they are combined with backend services, dashboard visualization, and explainability support. The developed prototype can serve as a foundation for future work in real-time network monitoring, security analytics, and AI-assisted incident response. Future experiments can focus on live traffic collection, model retraining, database integration, deployment with containers, and testing the system on different network datasets.

Overall, the results support the main objective of the project. The proposed system does not only classify network traffic but also presents the results in an analyst-oriented interface. This makes the project relevant not only as a machine learning experiment but also as a practical software prototype for AI-powered intrusion detection.

The limitations of the current system and possible future improvements are summarized in Table 4.3.

Table 4.3: Limitations and Future Improvements

Limitation	Possible Improvement
Real-time monitoring is simulated using pre-processed records	Integrate live packet capture tools such as CICFlowMeter, Zeek, or packet sniffing modules
Dataset-specific evaluation	Test the model on different IDS datasets and real network traffic
In-memory event history	Add persistent database storage
Limited dashboard functions	Add authentication, notifications, and advanced filtering
Temporal performance degradation	Apply periodic retraining and model drift monitoring
Weak performance on minority web attack classes	Use more balanced data, additional features, or specialized web attack models
Limited federated learning experiment	Extend the current FedAvg-based simulation with non-IID client distributions, larger client populations, communication-cost analysis, and enhanced minority-class handling

5. CONCLUSION

This project presented an AI-powered real-time-like network attack detection system and analyst dashboard. The main objective was to develop a machine learning-based intrusion detection prototype that can classify network traffic, detect malicious activities, identify attack types, and present the results through an analyst-oriented dashboard.

The project successfully combined machine learning model development with software system implementation. In the machine learning part, network traffic data was cleaned, analyzed, and transformed through preprocessing, feature engineering, leakage analysis, and model training steps. Binary and multiclass classification models were developed to detect attacks and classify attack categories. Different models, including Random Forest, XGBoost, and CatBoost, were compared using evaluation metrics such as accuracy, weighted F1-score, macro F1-score, ROC-AUC, confusion matrices, and per-class F1-score.

The experimental results showed that the proposed approach achieved strong performance in the per-file evaluation scenario. The binary classification model was able to distinguish benign and malicious traffic with high accuracy and F1-score values. The multiclass classification model also achieved strong results for most major attack categories. However, the results also showed that minority web attack classes, such as Web Attack - XSS and Web Attack - Brute Force, were more difficult to classify. This finding shows the importance of using per-class evaluation instead of relying only on overall accuracy.

The results also demonstrated that time-based evaluation is more challenging than per-file evaluation. This is an important observation because real network traffic changes over time, and models may face different traffic patterns during deployment. Therefore, future work should focus more on temporal generalization, model drift monitoring, and retraining strategies.

In the software implementation part, the trained models were exported and integrated into a FastAPI backend. The backend provided prediction, history, statistics, and explainability outputs to the frontend dashboard. The dashboard visualized traffic flow, attack distribution, recent security events, and SHAP-based explanation information. This showed that the trained machine learning models can be transformed from offline notebook experiments into a working intrusion detection prototype.

The project also highlighted the importance of explainability in AI-based security systems. By displaying feature-based explanations, the system helps analysts understand why a traffic record is classified as malicious. This improves the transparency of the model and supports analyst decision-making during alert investigation.

In addition to the centralized intrusion detection pipeline, a FedAvg-based federated learning

simulation was conducted using five distributed clients. The federated model achieved strong holdout performance while training was performed without sharing raw client data. Although the centralized Random Forest models achieved higher multiclass performance, the federated experiment demonstrated the feasibility of privacy-preserving distributed intrusion detection using collaborative model training.

Overall, the project achieved its main goal by developing an end-to-end AI-powered intrusion detection prototype. The system does not only provide classification results but also presents them in a practical dashboard environment. This makes the project relevant both as a machine learning study and as a software engineering implementation for cybersecurity monitoring.

Future improvements can extend the current prototype into a more realistic and production-ready system [6], [8], [9]. Possible future work includes:

- **Live packet capture integration:** Integrating tools such as CICFlowMeter, Zeek, or packet sniffing modules to enable real-time traffic ingestion instead of using preprocessed records.
- **Testing on different datasets:** Evaluating the model on different IDS datasets and real network traffic environments to assess generalization capability.
- **Persistent database storage:** Replacing the current in-memory event history with a permanent database to support long-term alert management.
- **User authentication and advanced filtering:** Adding authentication mechanisms, notification systems, and advanced alert filtering to improve dashboard functionality.
- **Model drift monitoring and retraining:** Applying periodic retraining strategies and drift detection mechanisms to maintain detection performance over time.
- **Advanced federated learning experiments:** Extending the current FedAvg-based federated learning simulation with non-IID client distributions, larger client populations, client selection strategies, communication-cost analysis, and improved minority-class handling to better represent realistic distributed intrusion detection environments.
- **Improved minority class detection:** Using more balanced training data, additional features, or specialized models to improve performance on minority web attack classes such as Web Attack - XSS and Web Attack - Brute Force.
- **SIEM integration:** Connecting the system to Security Information and Event Management platforms to support enterprise-level security operations.
- **Container-based deployment:** Deploying the system using Docker or similar containerization tools to improve portability and scalability.

With these improvements, the proposed system can be further developed into a more robust AI-assisted network security monitoring solution that is suitable for real-world deployment environments.

REFERENCES

- [1] D. Gümüşbaş, T. Yıldırım, A. Genovese, and F. Scotti, “A comprehensive survey of databases and deep learning methods for cybersecurity and intrusion detection systems,” *IEEE Systems Journal*, vol. 15, no. 2, pp. 1717–1731, 2021. DOI: 10.1109/JSYST.2020.2992966
- [2] J. Lansky, S. Ali, M. Mohammadi, M. K. Majeed, S. H. T. Karim, S. Rashidi, M. Hosseinzadeh, and A. M. Rahmani, “Deep learning-based intrusion detection systems: A systematic review,” *IEEE Access*, vol. 9, pp. 101 574–101 599, 2021. DOI: 10.1109/ACCESS.2021.3097247
- [3] F. M. Anis, M. Alabdullatif, S. Aljbli, and M. Hammoudeh, “A survey on the applications of deep learning in network intrusion detection systems to enhance network security,” *IEEE Access*, vol. 13, pp. 185 357–185 373, 2025. DOI: 10.1109/ACCESS.2025.3624952
- [4] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, “Benchmarking of machine learning for anomaly based intrusion detection systems in the cids2017 dataset,” *IEEE Access*, vol. 9, pp. 22 351–22 370, 2021. DOI: 10.1109/ACCESS.2021.3056614
- [5] R. Singh and G. Srivastav, “Novel framework for anomaly detection using machine learning technique on cic-ids2017 dataset,” in *2021 International Conference on Technological Advancements and Innovations (ICTAI)*, 2021, pp. 632–636. DOI: 10.1109/ICTAI53825.2021.9673238
- [6] M. Cantone, C. Marrocco, and A. Bria, “Machine learning in network intrusion detection: A cross-dataset generalization study,” *IEEE Access*, vol. 12, pp. 144 489–144 508, 2024. DOI: 10.1109/ACCESS.2024.3472907
- [7] G. Engelen, V. Rimmer, and W. Joosen, “Troubleshooting an intrusion detection dataset: The cids2017 case study,” in *2021 IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009
- [8] M. El-Hajj, “Ai-powered threat detection and response: Leveraging machine learning for real-time intrusion detection systems (ids) using network traffic data,” in *2025 5th Intelligent Cybersecurity Conference (ICSC)*, 2025, pp. 84–90. DOI: 10.1109/ICSC65596.2025.11139927
- [9] S. Banerjee, B. Basak, S. Mandal, S. Manna, S. Chakraborty, S. Ghatak, and A. Das, “Real-time anomaly detection in network traffic: A hybrid ml-dl approach,” in *2025 International Conference on Engineering Innovations and Technologies (ICoEIT)*, 2025, pp. 536–539. DOI: 10.1109/ICoEIT63558.2025.11211805

- [10] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in nids datasets: A case study on cic-ids-2017 and cse-cic-ids-2018," in *2022 IEEE Conference on Communications and Network Security (CNS)*, 2022, pp. 254–262. DOI: 10.1109/CNS56114.2022.9947235
- [11] N. Mearaj and M. A. Wani, "Zero-day attack detection with machine learning and deep learning," in *2023 10th International Conference on Computing for Sustainable Global Development (INDIACom)*, 2023, pp. 719–725.
- [12] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "Rtids: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64 375–64 387, 2022. DOI: 10.1109/ACCESS.2022.3182333
- [13] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for iot security attacks," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 2545–2554, 2022. DOI: 10.1109/JIOT.2021.3077803
- [14] O. Aouedi, K. Piamrat, G. Muller, and K. Singh, "Fluids: Federated learning with semi-supervised approach for intrusion detection system," in *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*, 2022, pp. 523–524. DOI: 10.1109/CCNC49033.2022.9700632
- [15] O. Belarbi, T. Spyridopoulos, E. Anthi, I. Mavromatis, P. Carnelli, and A. Khan, "Federated deep learning for intrusion detection in iot networks," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 237–242. DOI: 10.1109/GLOBECOM54140.2023.10437860
- [16] K. Roshan and A. Zafar, "Using kernel shap xai method to optimize the network anomaly detection model," in *2022 9th International Conference on Computing for Sustainable Global Development (INDIACom)*, 2022, pp. 74–80. DOI: 10.23919/INDIACom54597.2022.9763241
- [17] Z. Ali, W. Tiberti, A. Marotta, and D. Cassioli, "A lightweight intrusion detection system for iot based on deep transfer learning at the edge," in *2025 IEEE 9th Forum on Research and Technologies for Society and Industry (RTSI)*, 2025, pp. 1–6. DOI: 10.1109/RTSI64020.2025.11212527
- [18] M. Mbow, H. Koide, and K. Sakurai, "An intrusion detection system for imbalanced dataset based on deep learning," in *2021 Ninth International Symposium on Computing and Networking (CANDAR)*, 2021, pp. 38–47. DOI: 10.1109/CANDAR53791.2021.00013

DECLARATION

Bu projenin tasarımı, hazırlanması, yürütülmesi, araştırmalarının yapılması ve bulgularının analizlerinde bütün bilgilerin etik davranış ve akademik kurallar çerçevesinde elde edilerek sunulduğunu; ayrıca şablonda yer alan yazım kurallarına uygun olarak hazırlanan bana ait olmayan her türlü ifade ve bilginin kaynağına eksiksiz atıf yapıldığını, bilimsel etiğe uygun olarak kaynak gösterildiğini bildirir ve taahhüt ederim.

COMMITMENT

In the design, preparation, execution, research and analysis of the findings of this project, all information is obtained and presented within the framework of ethical behavior and academic rules; I also declare and undertake that all statements and information that are prepared in accordance with the spelling rules in the template and that do not belong to me are cited in full, and that the source is cited in accordance with scientific ethics.

Göktuğ VARAN 210408029
Mehmet Taha YILMAZ 210408039

İmza